

Image filters and its Applications

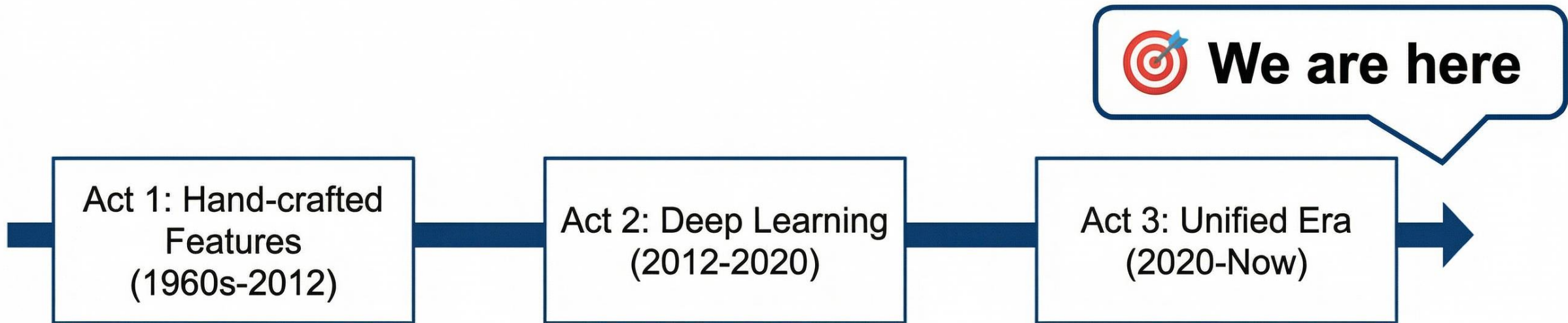
Instructor: Hao WANG
AI Thrust, Information Hub



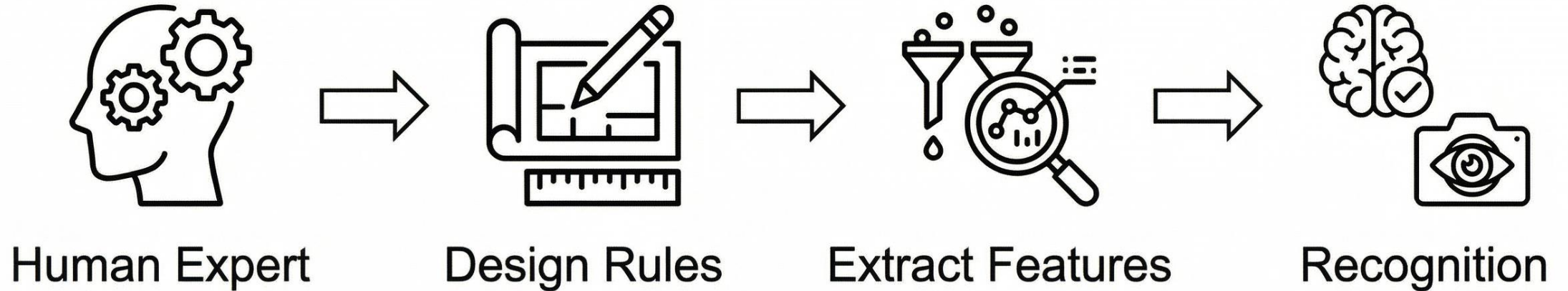
Recap of Last Week



Three Eras of CV



Hand-crafted Approach



Philosophy: Humans tell machines

Examples: SIFT, HOG, Haar

Limitations

✗ Task-specific

✗ Poor generalization

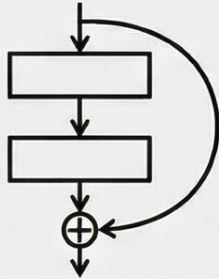
✗ Hard to design

✗ Performance ceiling

Breakthrough needed...

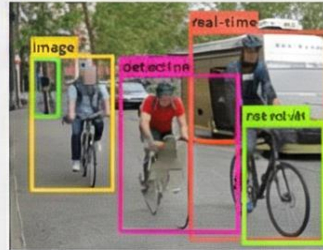


DL Achievements (2012-2020)



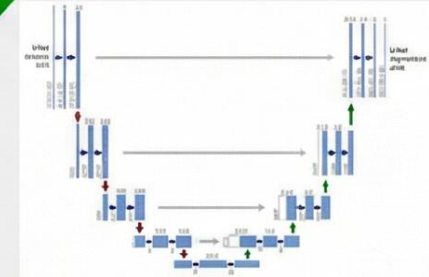
ResNet:

Classification: 95%+



YOLO:

Detection: Real-time



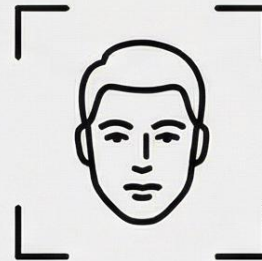
U-Net:

Segmentation: Pixel-level



OpenPose:

Pose: Joints



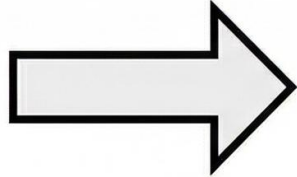
FaceNet:

Face: Beyond human

+ Depth, Video, 3D...

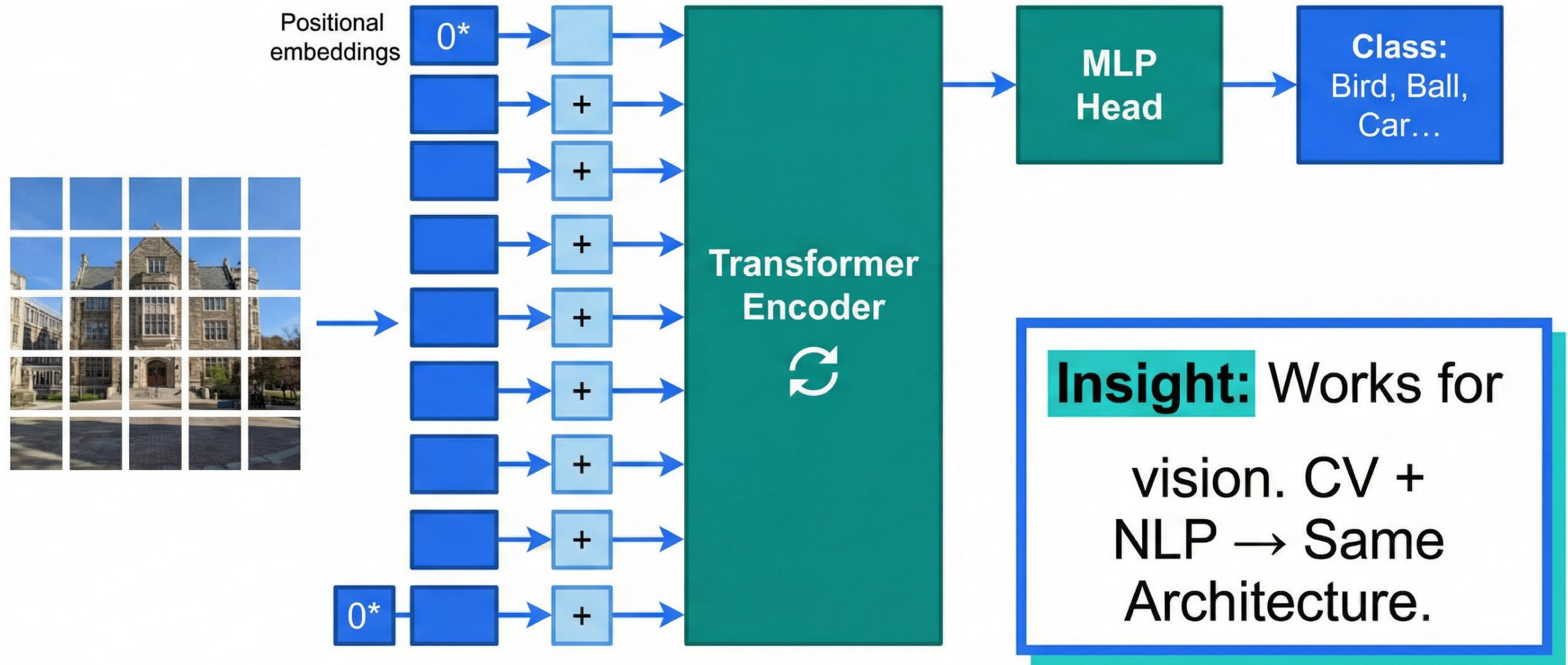
Act 2 Limitations

- ✗ One model, one task
- ✗ Understanding $\neq$ Generation
- ✗ Massive labeled data
- ✗ Lacks generalization

Can we unify them? 



Vision Transformer (2020)



Four Unifying Trends

1 Multimodal
Text + Vision

2 Zero-shot
No training

3 Visual Dialog
Reasoning

4 World Model
Unified



Image = Matrix



=

255	200	255
200	50	200
255	200	255

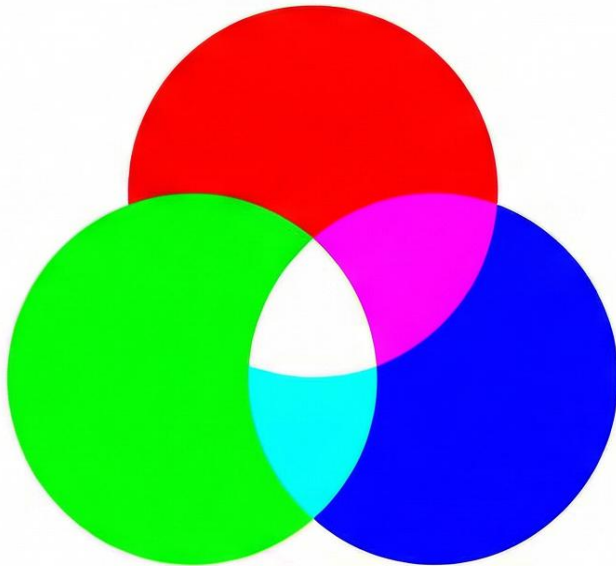
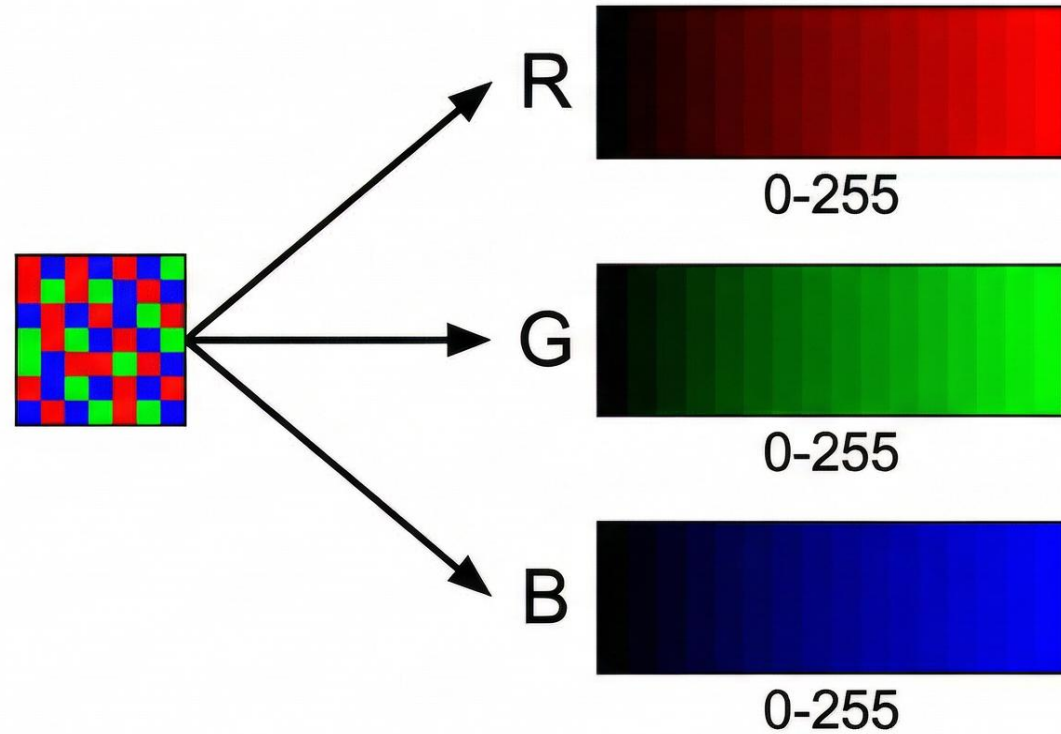
Each pixel = number
(grayscale) or 3
numbers (color)

Range: 0-255

Everything is
numbers.

Color = 3 Channels (RGB)

Color = 3 Channels
Each pixel = (R, G, B)
Each: 0-255



→ Red + Green + Blue → All colors

Quiz

Why is the Transformer considered a foundational model of modern AI? Which of the following best explains the *primary reason* for its success?

- A. It naturally supports multimodal inputs such as text, images, and audio.
- B. It models long-range dependencies better than previous architectures.
- C. It exhibits strong and predictable scaling behavior with more data, parameters, and compute.
- D. Its attention mechanism closely resembles human cognitive processes.



Today's Plan

Image filters and its applications!



Who is this person?

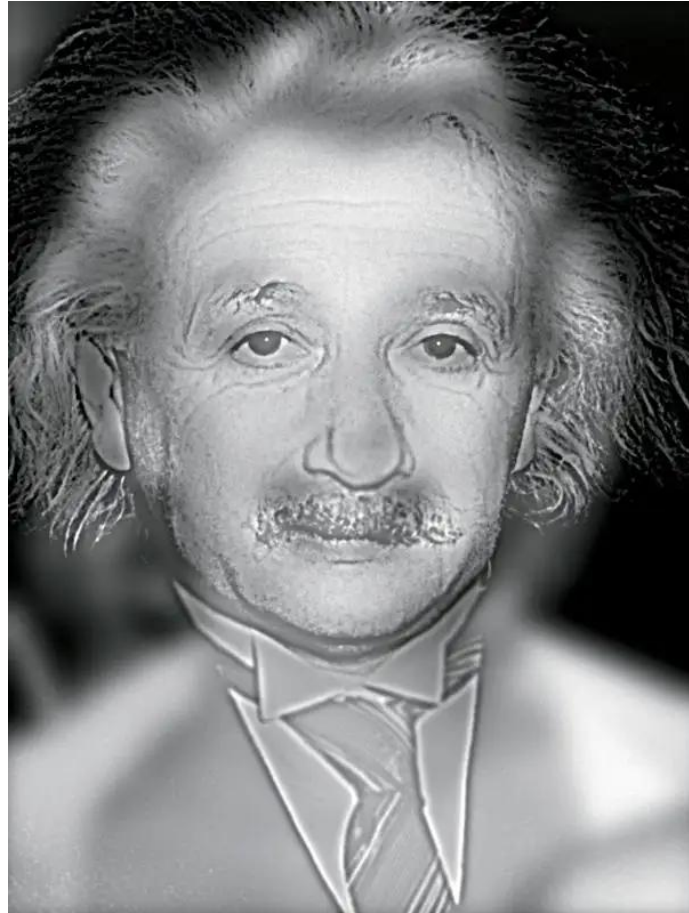


Image filtering

- Image filtering: compute function of local neighborhood at each position
- Really important!
 - Enhance images
 - Denoise, resize, etc.
 - Extract information from images
 - Texture, edges, distinctive points, etc.
 - Detect patterns
 - Template matching
 - Future content: Convolutional Neural Networks!



Example: box filter

$g[\cdot, \cdot]$

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

“Mean Filter”: Every pixel in the neighborhood is treated with **equal importance**.



Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20						

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



$$g[\cdot, \cdot]^{\frac{1}{9}}$$

$$h[\cdot, \cdot]$$
[illegible]

Credit: S. Seitz

Image filtering

$$g[\cdot, \cdot] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				
						?			
				50					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	

Informally, what does the filter do?

0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1



Smoothing with box filter



Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

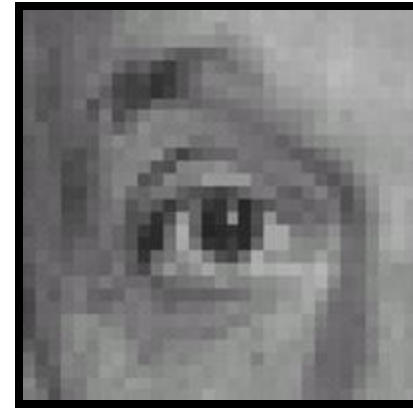
?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)



Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel



Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

—

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

(Note that filter sums to 1)



Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

—

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

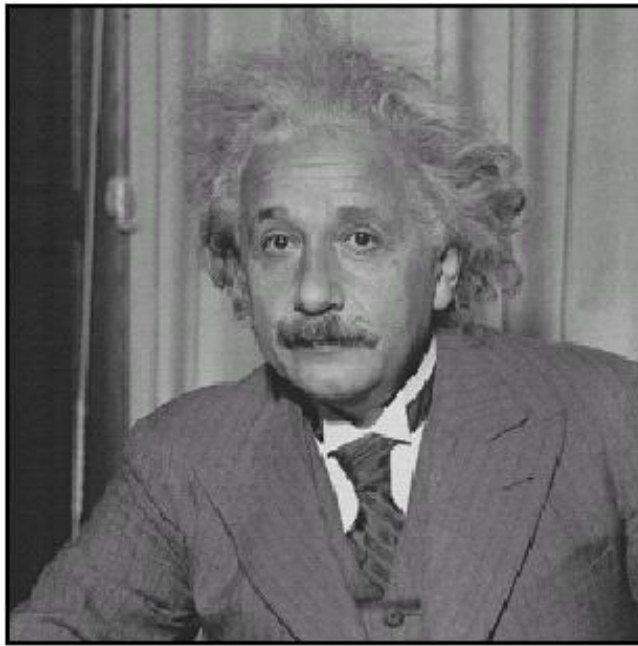


Sharpening filter

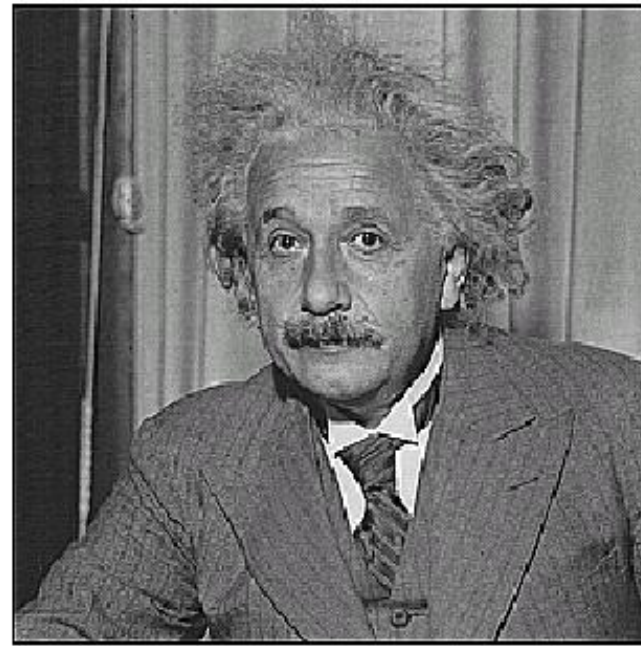
- Accentuates differences with local average



Sharpening



before



after

Source: D. Lowe



Key properties of linear filters

Linearity:

$$\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$$

Shift invariance: same behavior regardless of pixel location

$$\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$$

Any linear, shift-invariant operator can be represented as a convolution



More properties

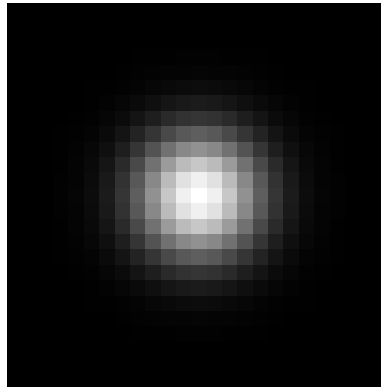
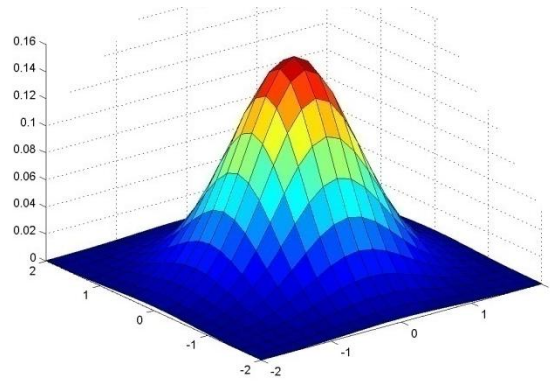
- Commutative: $a * b = b * a$
 - Conceptually no difference between filter and signal (image)
- Associative: $a * (b * c) = (a * b) * c$
 - Often apply several filters one after another: $((a * b_1) * b_2) * b_3$
 - This is equivalent to applying one filter: $a * (b_1 * b_2 * b_3)$
- Distributes over addition: $a * (b + c) = (a * b) + (a * c)$
- Scalars factor out: $ka * b = a * kb = k(a * b)$
- Identity: unit impulse $e = [0, 0, 1, 0, 0]$,
 $a * e = a$



Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness

It uses a weighted average where the **center pixel has the most influence**, and the influence of surrounding pixels decreases as they get further from the center.



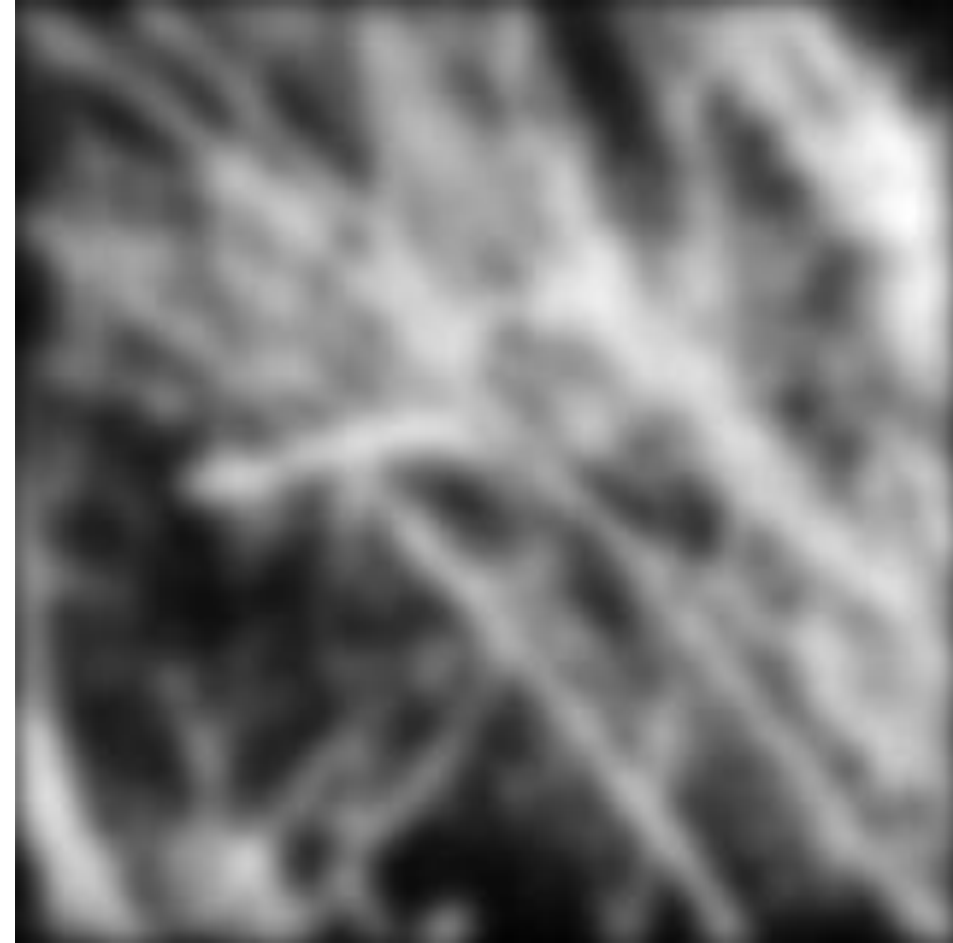
0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



Smoothing with Gaussian filter



Gaussian filters

- Remove “**high-frequency**” components from the image (low-pass filter)
 - Images become more smooth
- Convolution with self is another Gaussian
 - So can smooth with small-width kernel, repeat, and get same result as larger-width kernel would have
 - Convoluting two times with Gaussian kernel of width σ is same as convoluting once with kernel of width $\sigma\sqrt{2}$
- *Separable* kernel
 - Factors into product of two 1D Gaussians

More natural

More Flexible

More Efficient



Separability of the Gaussian filter

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian



Separability example

$$O(M \cdot N \cdot m \cdot n)$$

2D filtering
(center location only)

1	2	1
2	4	2
1	2	1

 $\ast$

2	3	3
3	5	5
4	4	6

$$O(M \cdot N \cdot (m + n))$$

The filter factors
into a product of 1D
filters:

1	2	1
2	4	2
1	2	1

 $=$

1
2
1

 $\times$

1	2	1
---	---	---

Perform filtering
along rows:

1	2	1
---	---	---

 $\ast$

2	3	3
3	5	5
4	4	6

 $=$

	11	
	18	
	18	

Followed by filtering
along the remaining column:

1
2
1

 $\ast$

	11	
	18	
	18	

 $=$

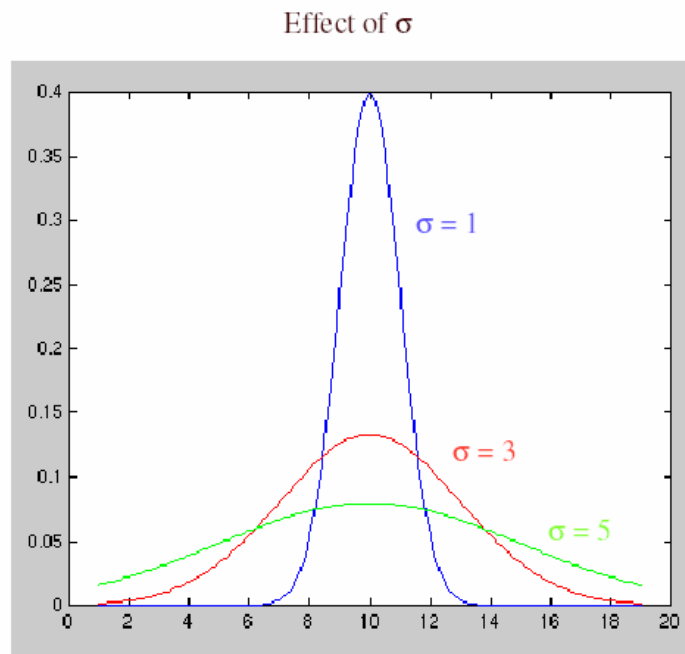
	65	



Practical matters

How big should the filter be?

- Values at edges should be near zero
- **Rule of thumb for Gaussian: set kernel half-width to $\geq 3\sigma$**



If $\sigma = 1$, your kernel radius should be 3, making the total kernel size 7×7 (3 pixels on each side + the center pixel).



Practical matters

- What about near the edge?
 - The filter window falls off the edge of the image
 - Need to extrapolate
 - Methods:
 - **Clip filter (black)**: Treat all pixels outside the image boundary as zero (black).
 - **Wrap around**: Treat the image as if it were a torus, where the right edge connects to the left edge.
 - **Copy edge**: Extend the value of the last pixel at the edge outward.
 - **Reflect across edge**: Mirror the image pixels across the boundary to fill the empty space.



Source: S. Marschner



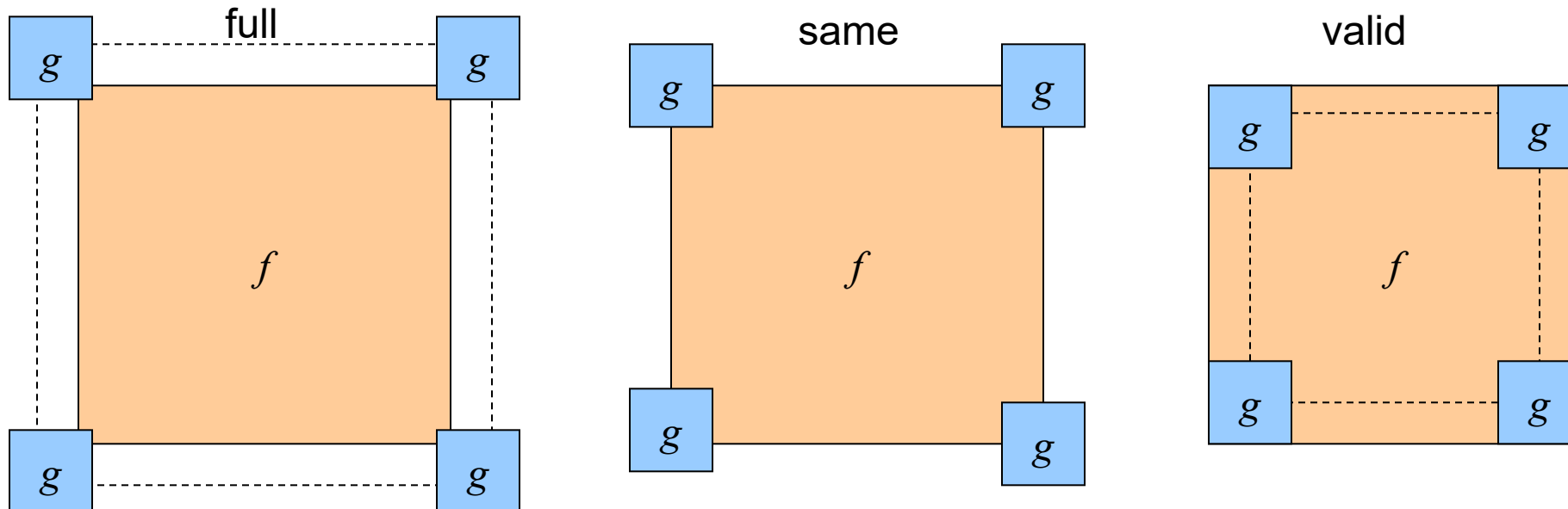
Practical matters

- Methods (Python):
 - clip filter (black): `convolve2d(f, g, boundary='fill', 0)`
 - wrap around: `convolve2d(f, g, boundary='wrap')`
 - reflect across edge: `convolve2d(f, g, boundary='symm')`

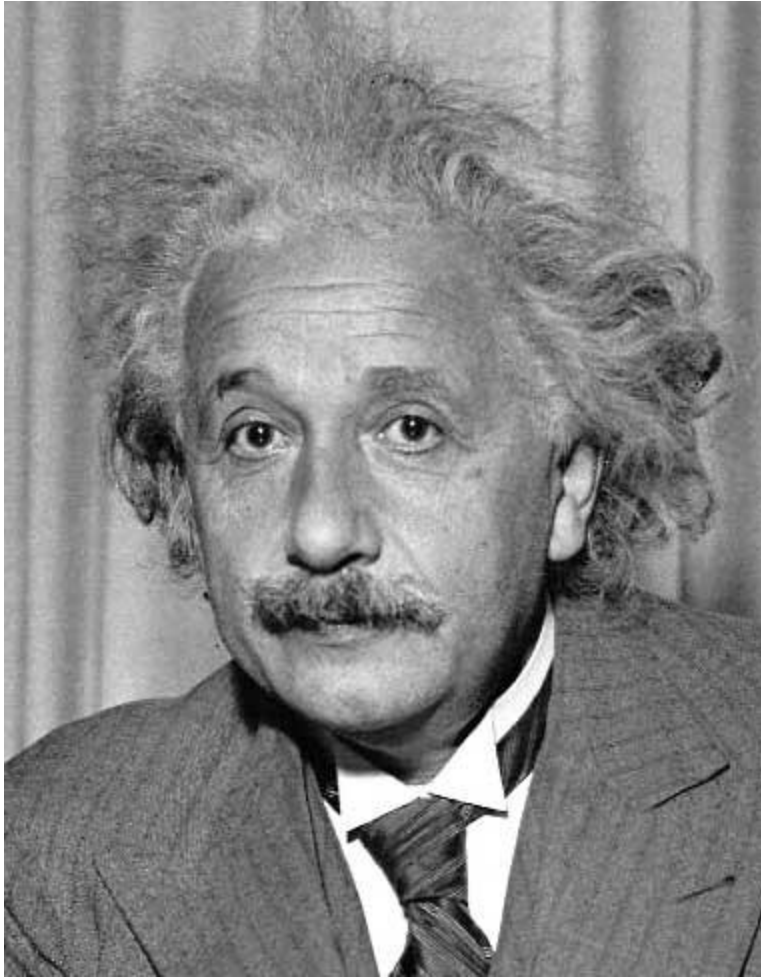


Practical matters

- What is the size of the output?
- Python: `convolve2d(g, f, mode)`
 - *mode* = 'full': output size is sum of sizes of *f* and *g*
 - *mode* = 'same': output size is same as *f*
 - *mode* = 'valid': output size is difference of sizes of *f* and *g*

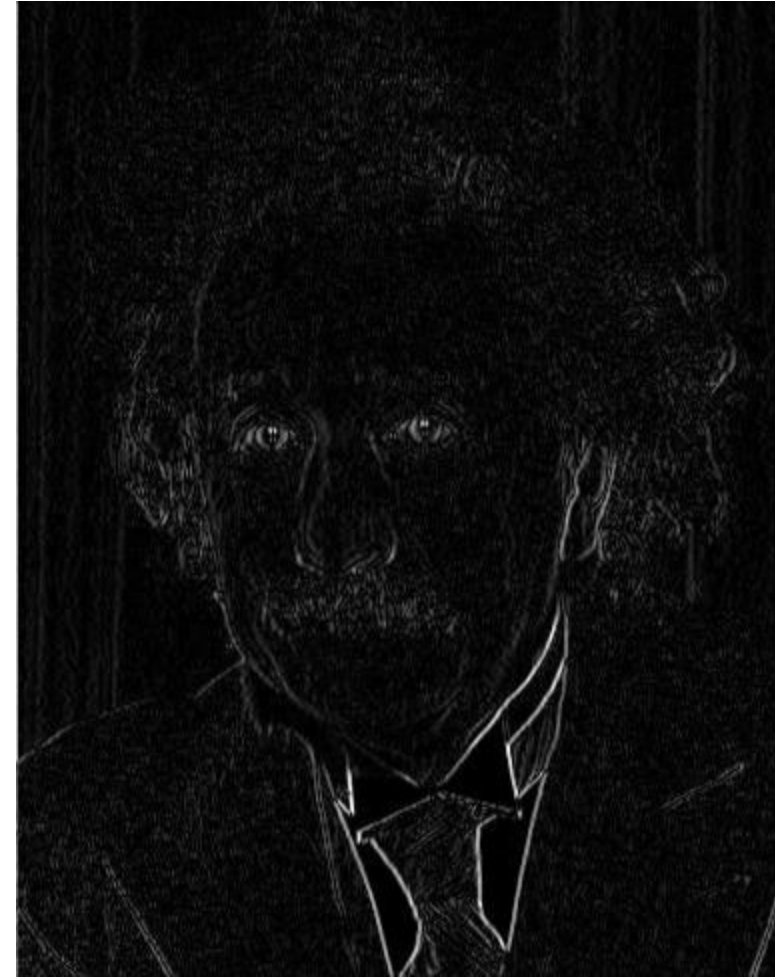


Other filters



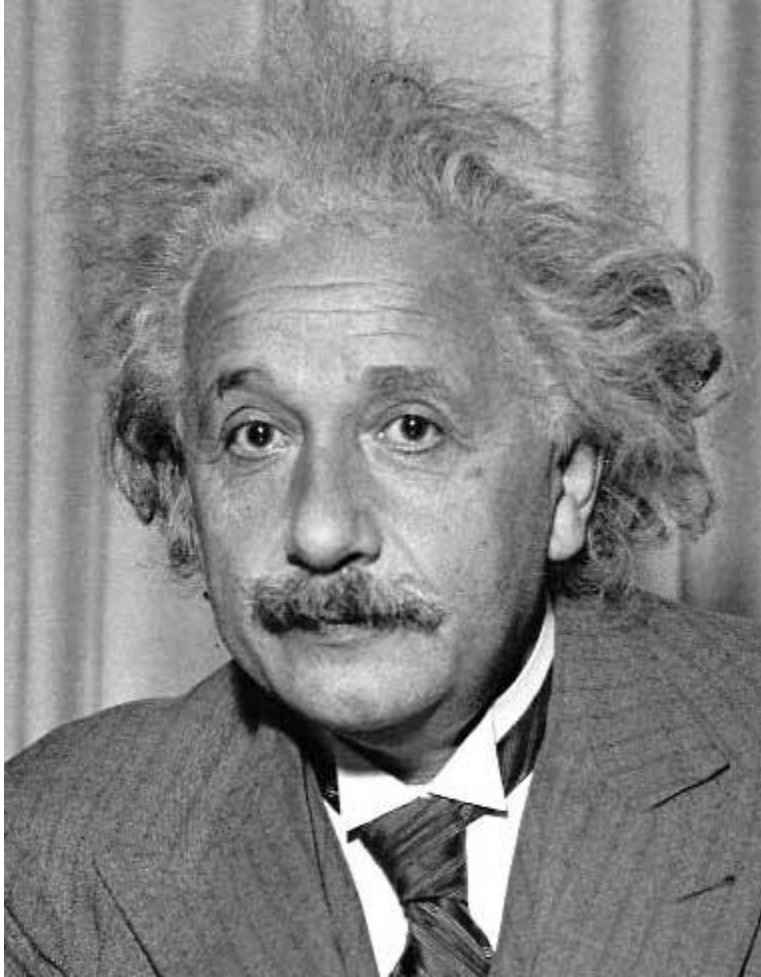
1	0	-1
2	0	-2
1	0	-1

Sobel



Vertical Edge
(absolute value)

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel

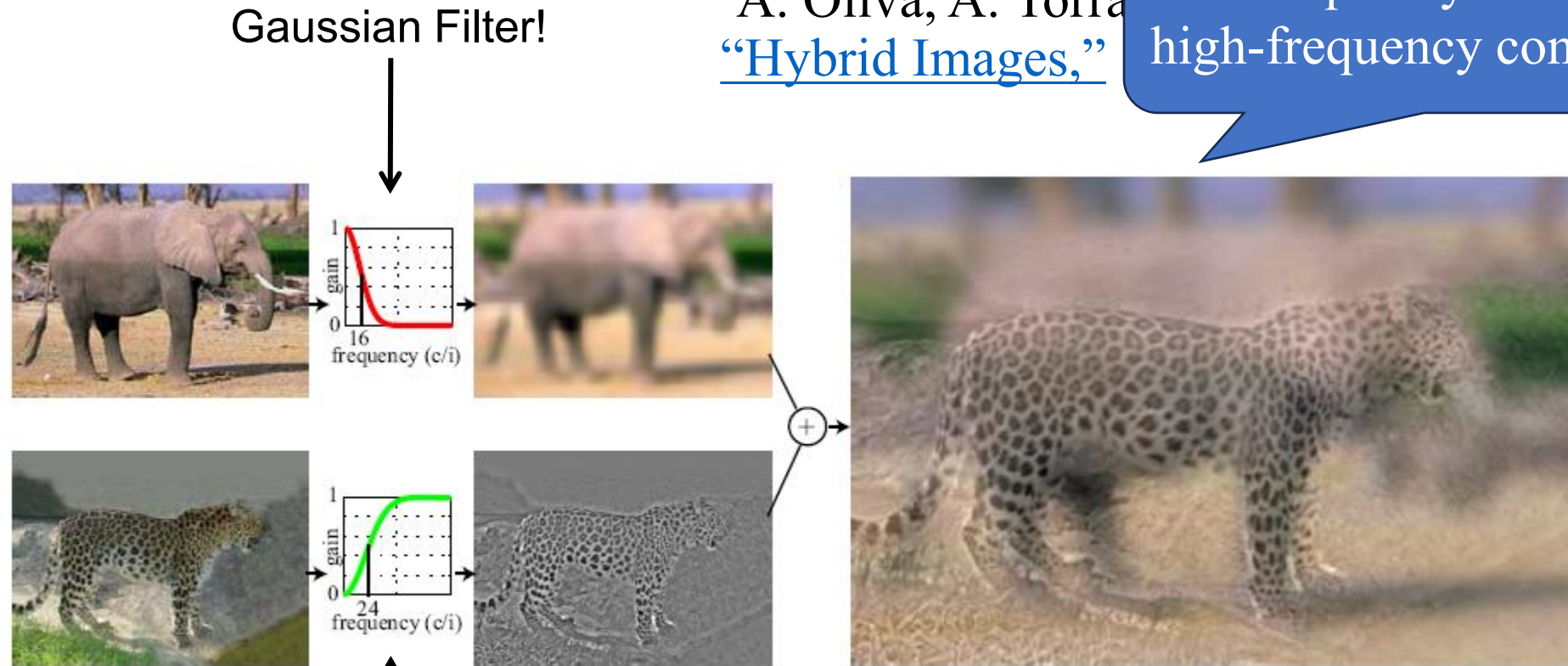


Horizontal Edge
(absolute value)

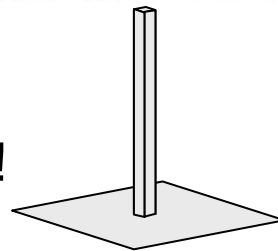
Application: Hybrid Images

A. Oliva, A. Torralba
“Hybrid Images,”

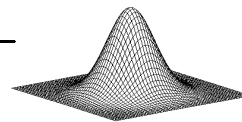
low-frequency components +
high-frequency components



Laplacian Filter!



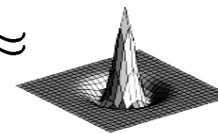
unit impulse



Gaussian

−

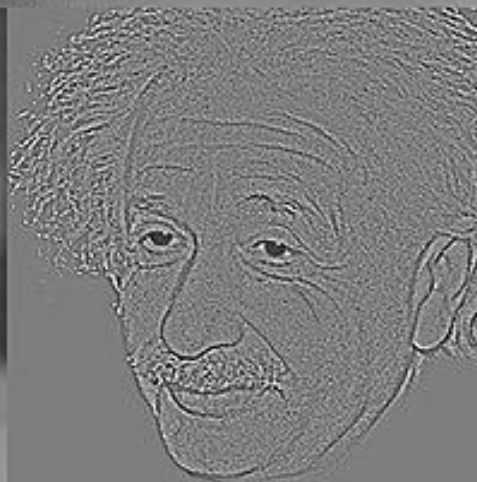
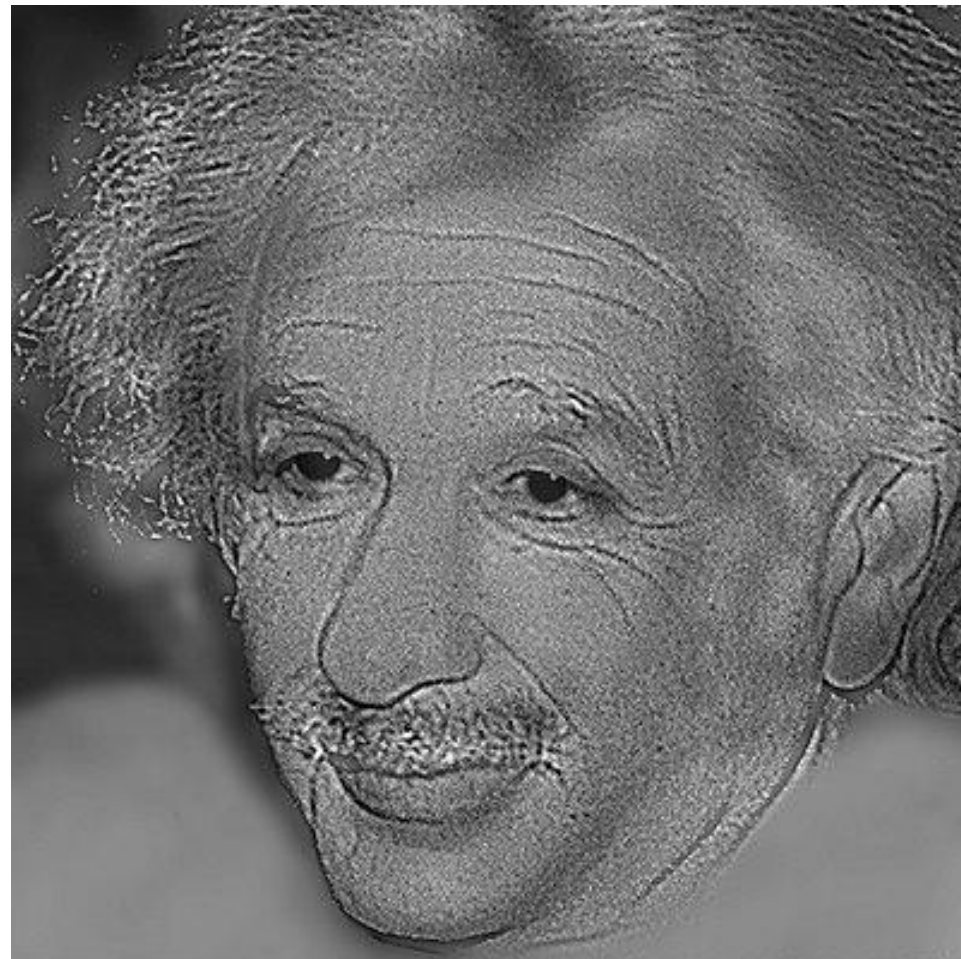
≈



Laplacian of Gaussian







https://en.wikipedia.org/wiki/Hybrid_image



Other Applications of Filters

- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid

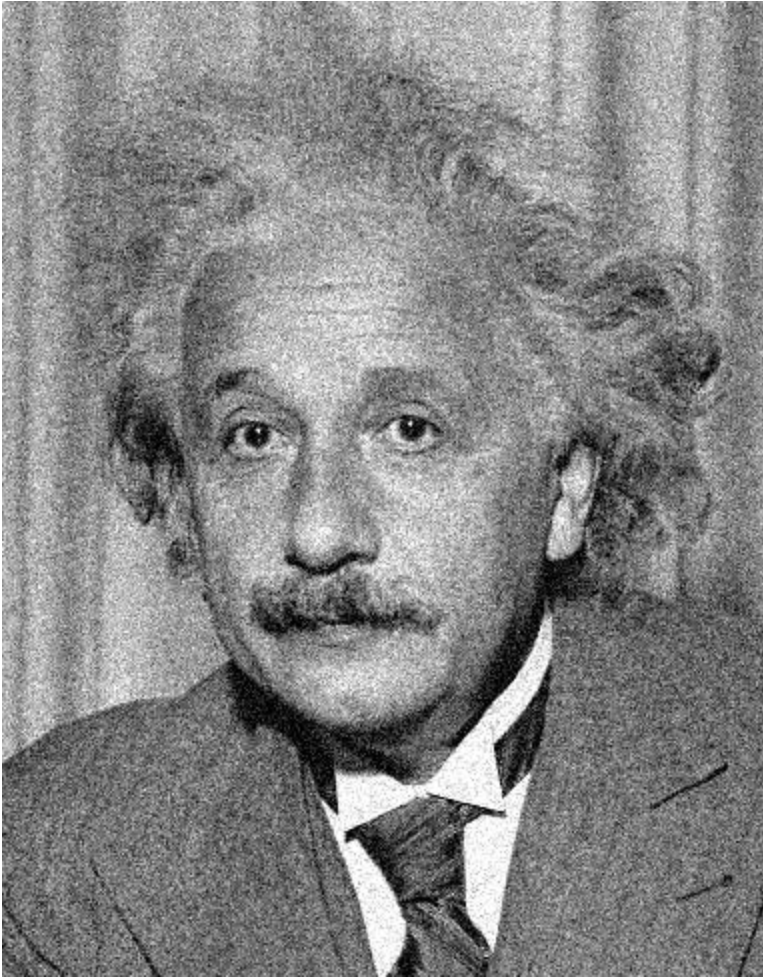


Other Applications of Filters

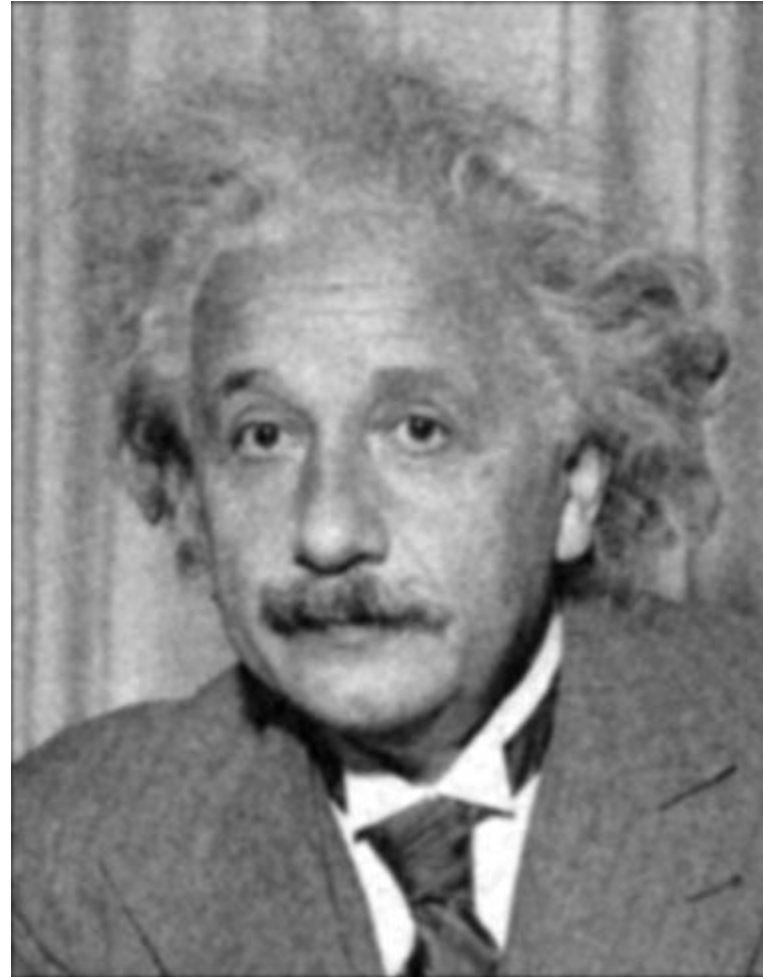
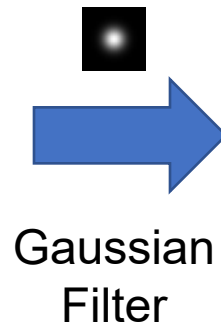
- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid



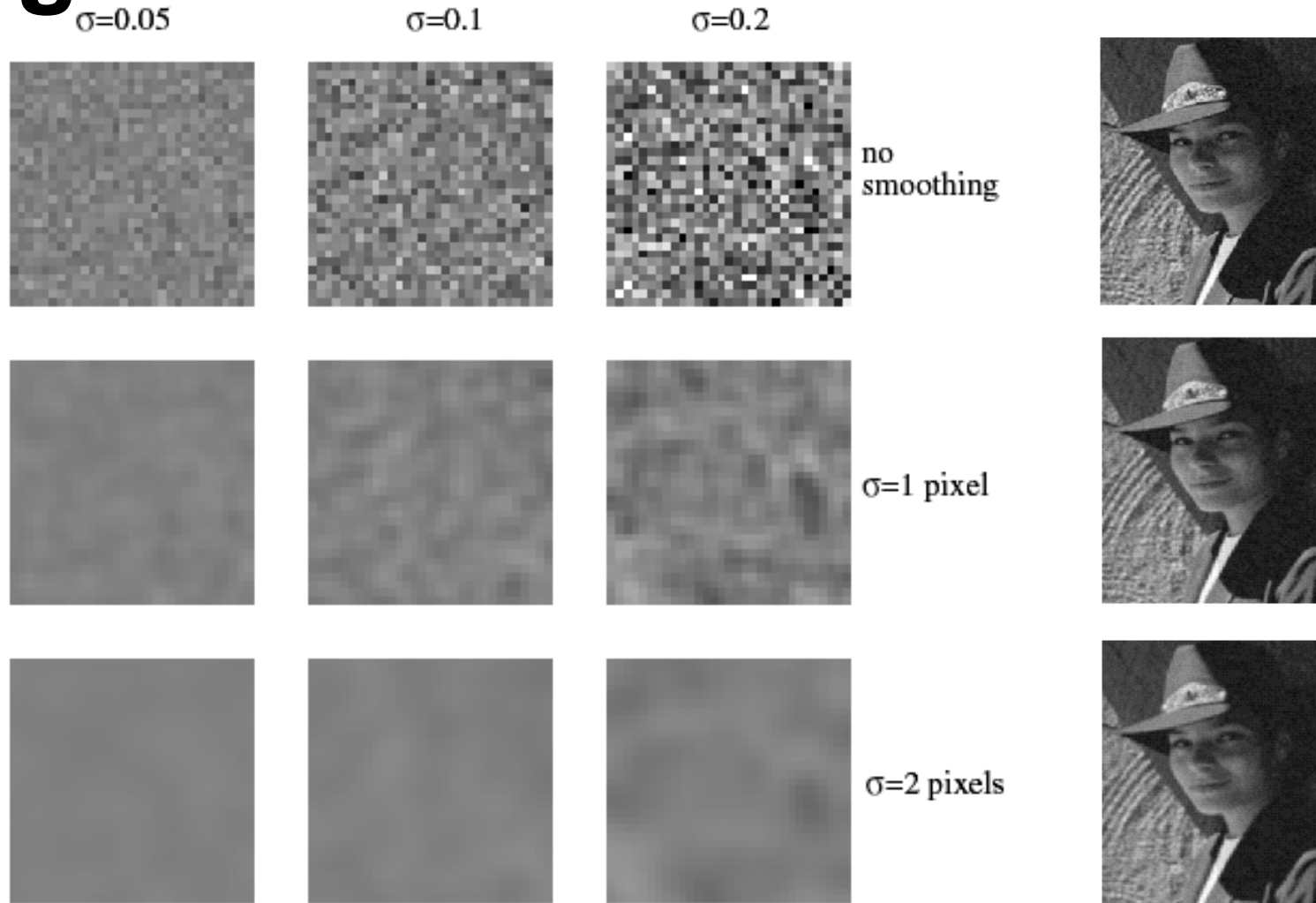
Denoising



Additive Gaussian Noise



Reducing Gaussian noise



Smoothing with larger standard deviations suppresses noise, but also blurs the image

Source: S. Lazebnik



Reducing salt-and-pepper noise (impulse noise) by Gaussian smoothing

3x3



5x5

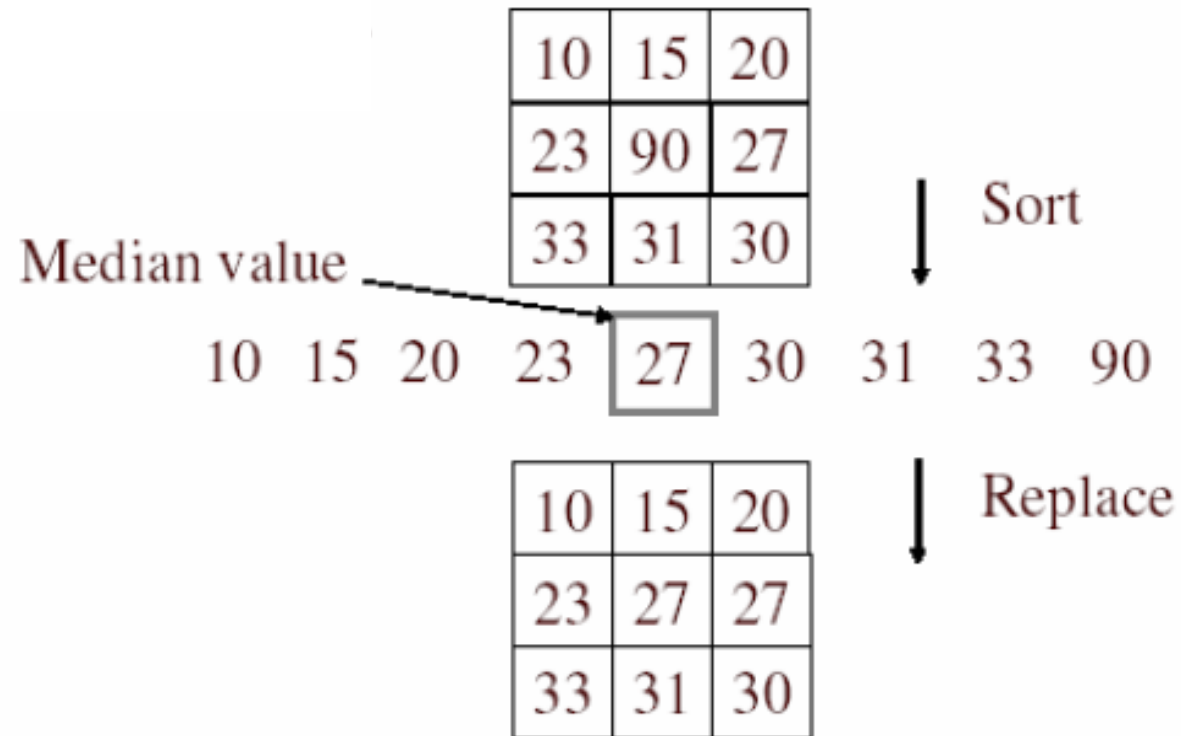


7x7



Alternative idea: Median filtering

- A **median filter** operates over a window by selecting the median intensity in the window



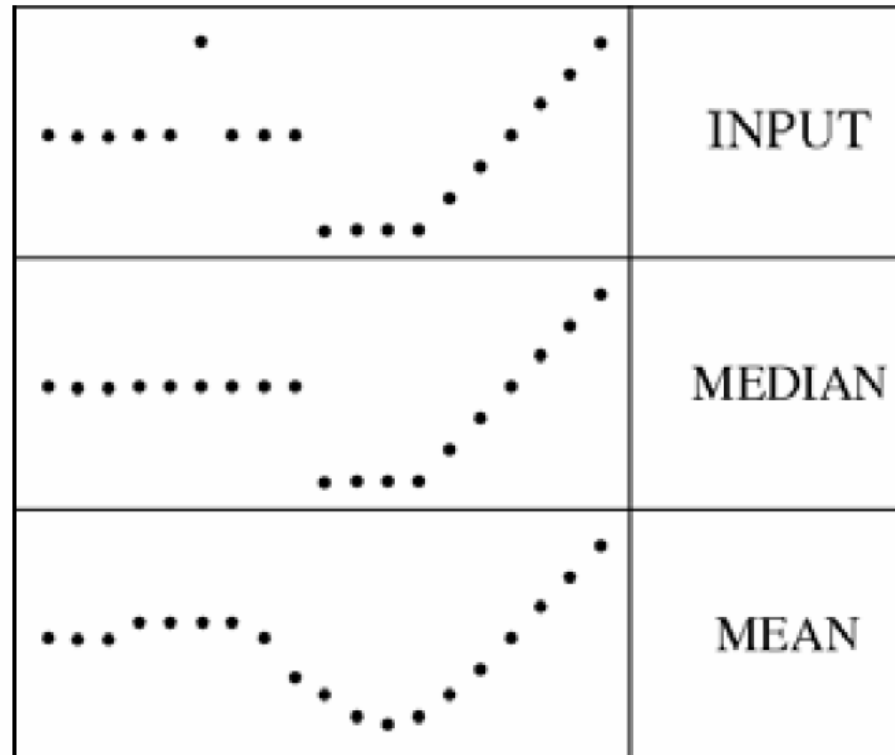
- Is median filtering linear?



Median filter

- What advantage does median filtering have over Gaussian filtering?
 - Robustness to outliers

filters have width 5 :

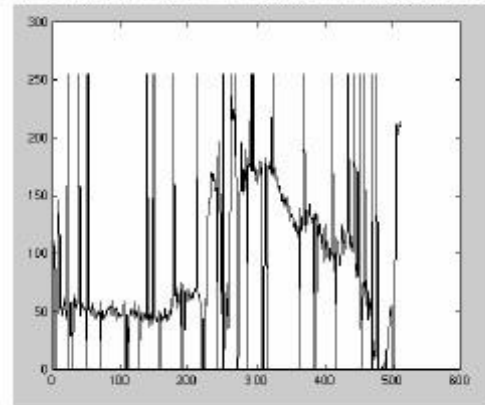
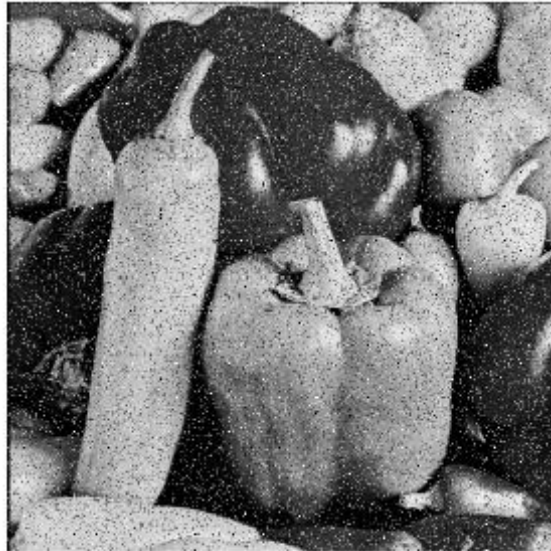


Source: K. Grauman

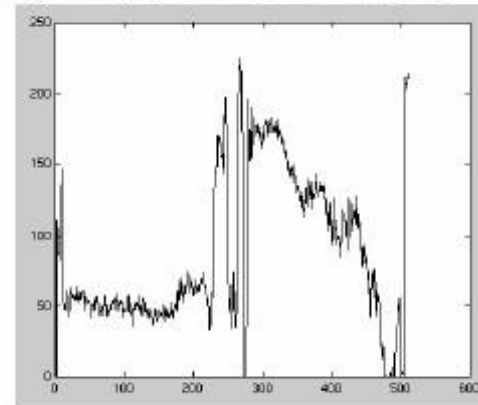


Median filter

Salt-and-pepper noise



Median filtered



Python: `scipy.ndimage.median_filter(image, size)`

Source: M. Hebert



Median Filtered Examples



original image



1px median filter



3px median filter



10px median filter

<http://en.wikipedia.org/wiki/File:Medianfilterp.png>
http://en.wikipedia.org/wiki/File:Median_filter_example.jpg



Median vs. Gaussian filtering

3x3

5x5

7x7

Gaussian



Median



Other filter choices

- Weighted median (pixels further from center count less)
- Clipped mean (average, ignoring few brightest and darkest pixels)
- Bilateral filtering (weight by spatial distance *and* intensity difference)

```
cv2.bilateralFilter(size, sigma_color, sigma_spatial)
```



Bilateral filtering

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

Diagram illustrating the components of the Bilateral Filter equation:

- $\frac{1}{W_p}$: Normalization Factor
- $G_{\sigma_s}(\|p - q\|)$: Space Weight (represented by a 2D Gaussian kernel image)
- $G_{\sigma_r}(|I_p - I_q|)$: Range Weight (represented by a 1D Gaussian kernel image)



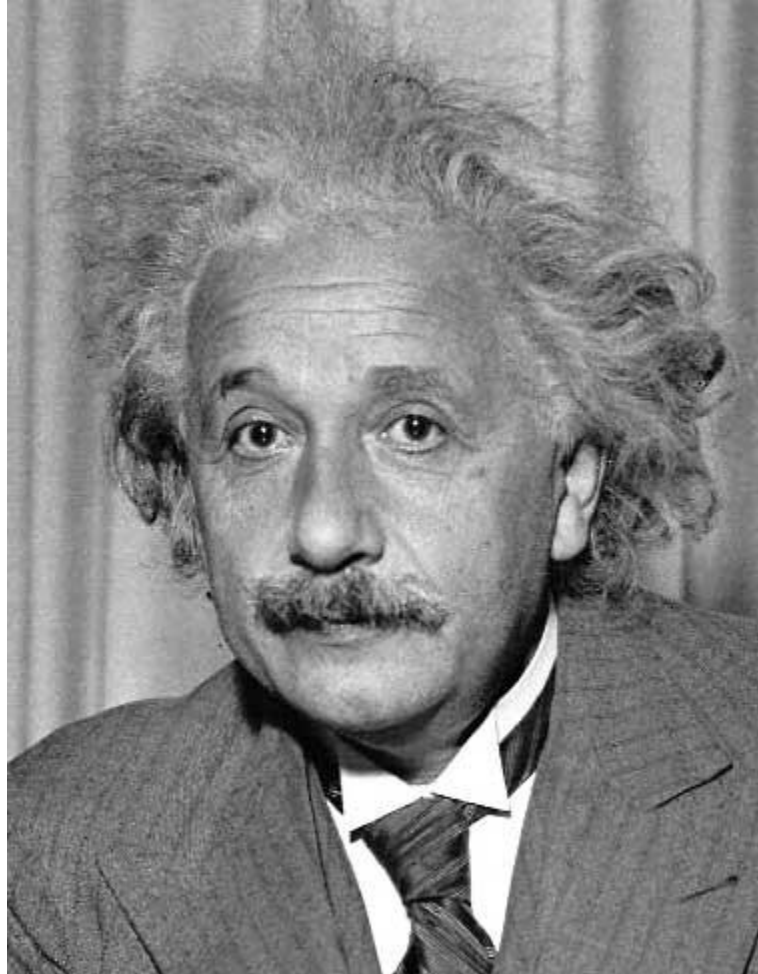
Outline: some applications of filtering

- Denoising
- Template matching
- Gaussian pyramid
- Laplacian pyramid



Template matching

- Goal: find  in image
- Main challenge: What is a good similarity or distance measure between two patches?



Matching with filters

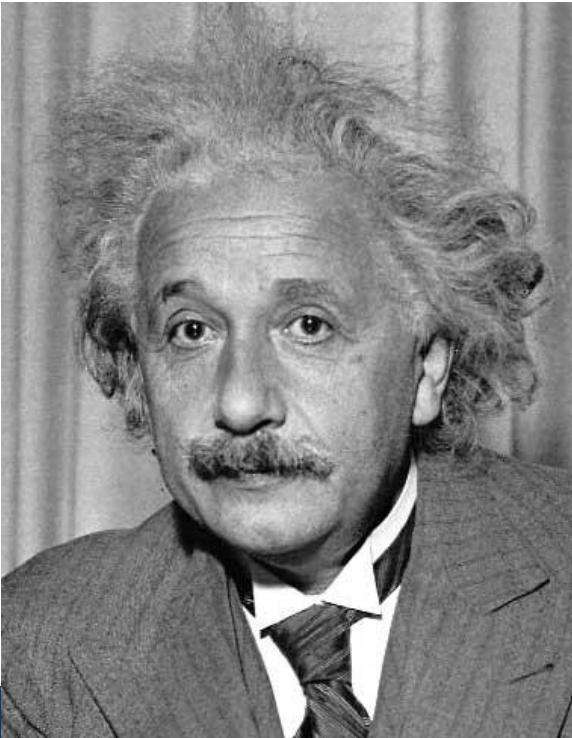
- Goal: find  in image
- Method 0: filter the image with eye patch

$$h[m,n] = \sum_{k,l} f[k,l] \text{ im}[m+k, n+l]$$

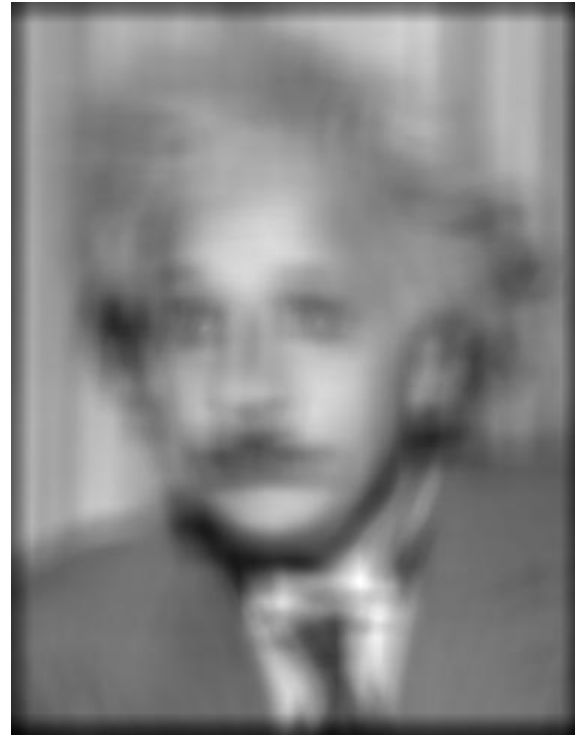
Brightness Bias: Areas that are generally bright will produce a high response regardless of whether they look like an eye, because large numbers multiplied by large numbers yield a large sum. This is why it looks like a bright, blurry version of the original face.

im = image
f = filter

What went wrong?



Input



Filtered Image



Matching with filters

- Goal: find  in image
- Method 1: filter the image with **zero-mean** eye

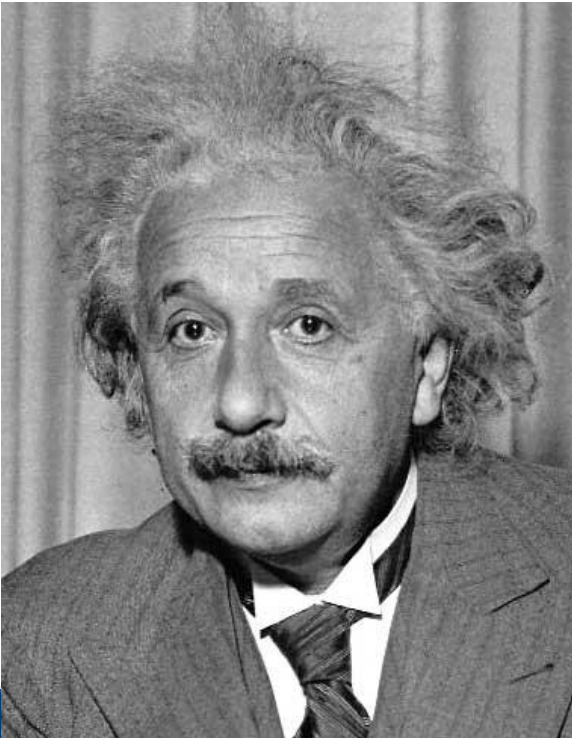
$$h[m, n] = \sum_{k, l} (f[k, l] - \bar{f}) \text{im}[m + k, n + l]$$

← mean of filter f

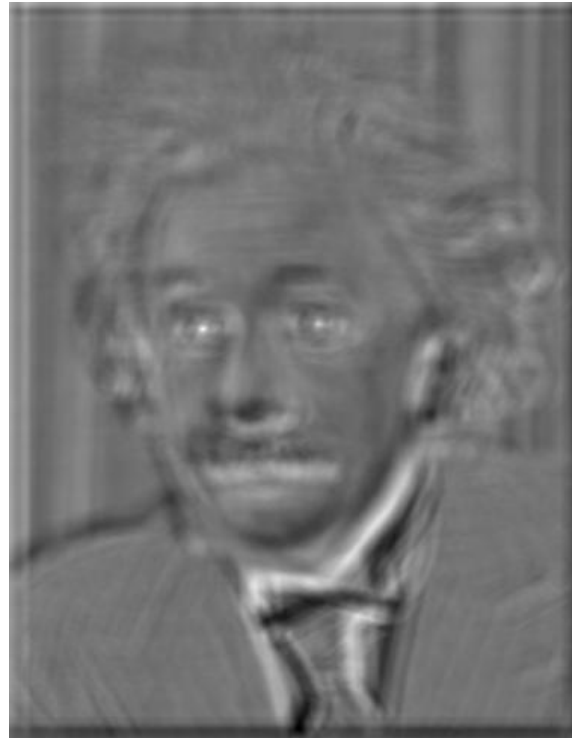
• **Pattern Over Brightness:** By subtracting the mean intensity ($\bar{f}$), the filter only gives a strong positive response when the *relative* patterns of light and dark match.

• **Canceling Uniform Areas:** In a perfectly flat, bright area (like a forehead), a zero-mean filter will sum to zero, effectively ignoring the area.

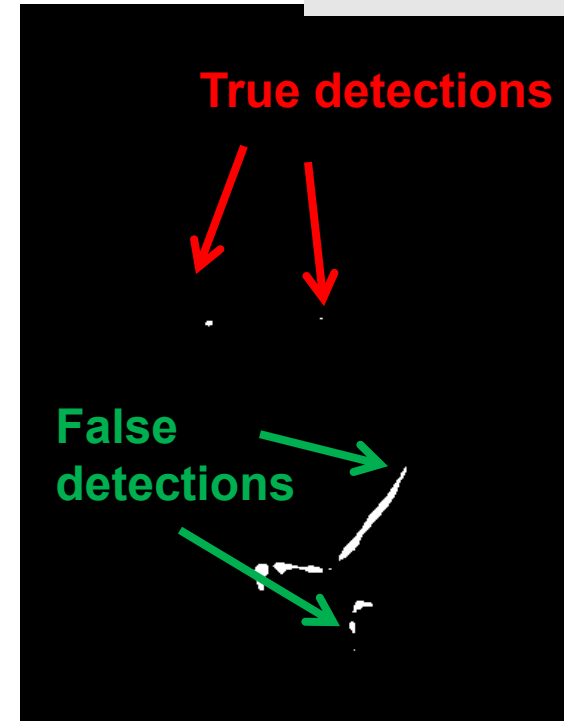
• **True Detections:** This allows the "Thresholded Image" to show sharp dots specifically at the eye locations, rather than glowing blobs across the whole face.



Input



Filtered Image (scaled)

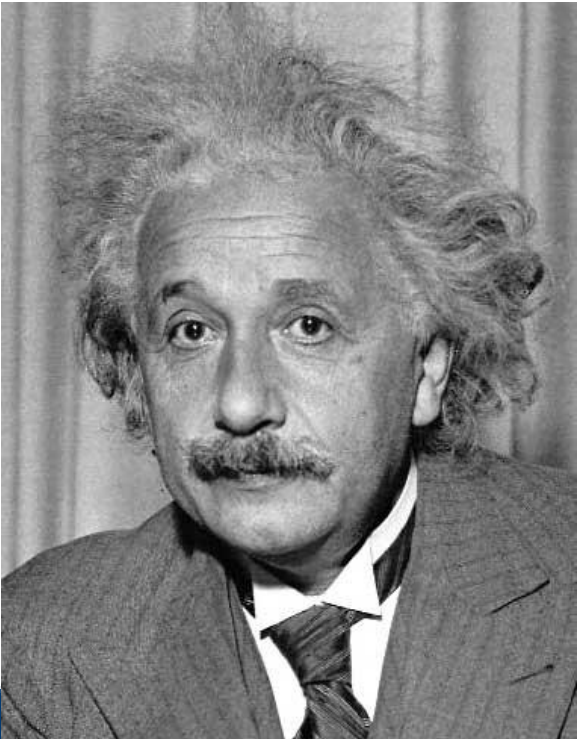


Thresholded Image

Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

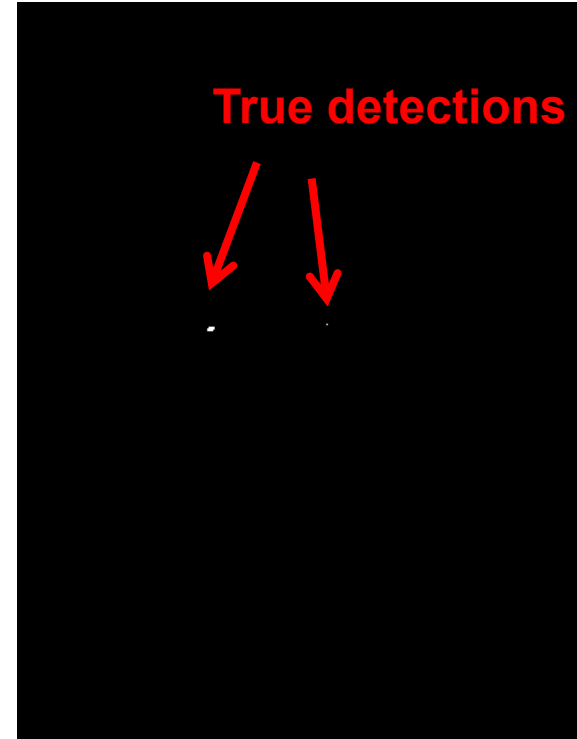
$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$



Input



1- sqrt(SSD)



Thresholded Image



Matching with filters

Can SSD be implemented with linear filters?

Filtering is much faster!

$$h[m,n] = \sum_{k,l} (f[k,l] - im[m+k,n+l])^2$$

$$h[m,n] = \sum_{k,l} (f[k,l]^2 - 2 \cdot im[m+k,n+l] \cdot f[k,l] + im[m+k,n+l]^2)$$

$$h[m,n] = \sum_{k,l} f[k,l]^2 - 2 \sum_{k,l} im[m+k,n+l] \cdot f[k,l] + \sum_{k,l} im[m+k,n+l]^2$$

$$h = \sum_{k,l} f[k,l]^2 - 2 \text{filter}(im, f) + \text{filter}(im.^2, \text{ones}(f.\text{shape}))$$

constant

linear filter

Element-wise square f, then sum
with ones kernel of size f

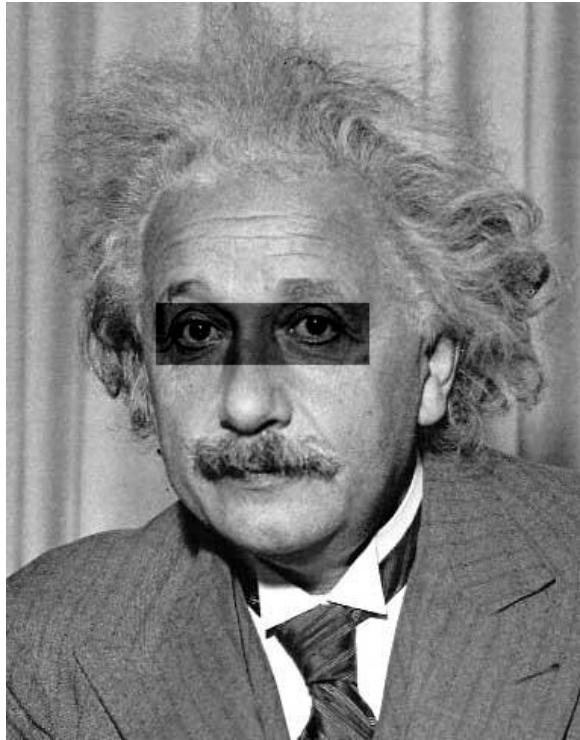


Matching with filters

- Goal: find  in image
- Method 2: Sum of Square Distance (SSD)

$$h[m, n] = \sum_{k, l} (f[k, l] - im[m + k, n + l])^2$$

What's the potential downside of SSD?



Input



1- sqrt(SSD)

Sensitive to intensity shift

Matching with filters

- Goal: find  in image
- Method 3: Normalized cross-correlation

$$h[m, n] = \frac{\sum (f - \bar{f})(im - \overline{im})}{\sqrt{\sum (f - \bar{f})^2 \sum (im - \overline{im})^2}}$$

Cosine Similarity!

$$h[m, n] = \frac{\begin{array}{c} \text{mean template} \\ \downarrow \\ \sum_{k,l} (f[k, l] - \bar{f}) \end{array} \begin{array}{c} \text{mean image patch} \\ \downarrow \\ \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}}) \end{array}}{\left(\sum_{k,l} (f[k, l] - \bar{f})^2 \sum_{k,l} (im[m + k, n + l] - \overline{im_{m,n}})^2 \right)^{0.5}}$$

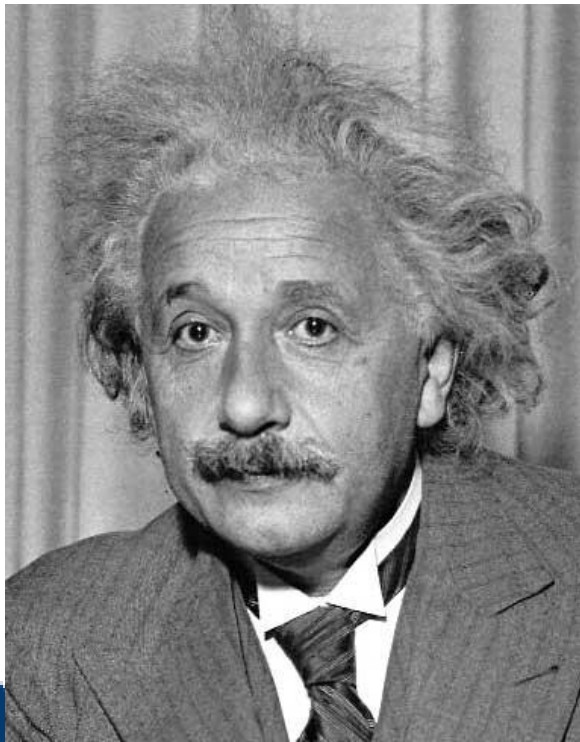
(divide by product of standard deviations of template and image patch)



Python: `cv2.matchTemplate(im, template, cv2.TM_CCOEFF_NORMED)`

Matching with filters

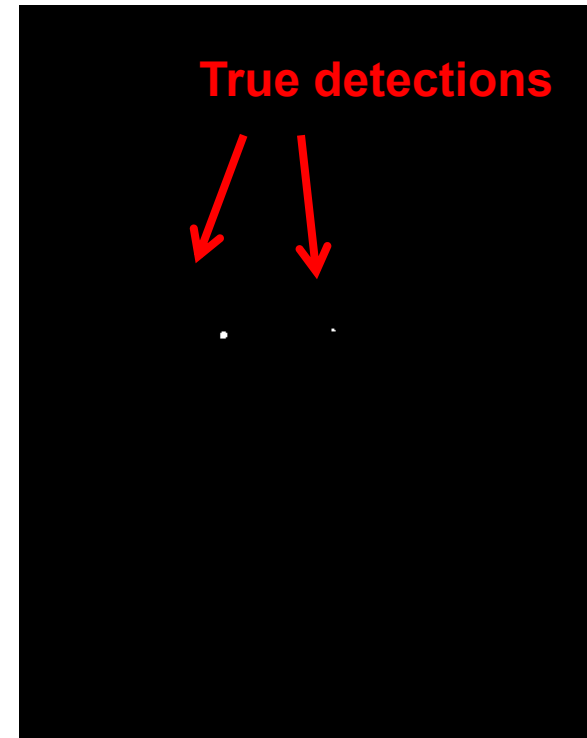
- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



Normalized X-Correlation

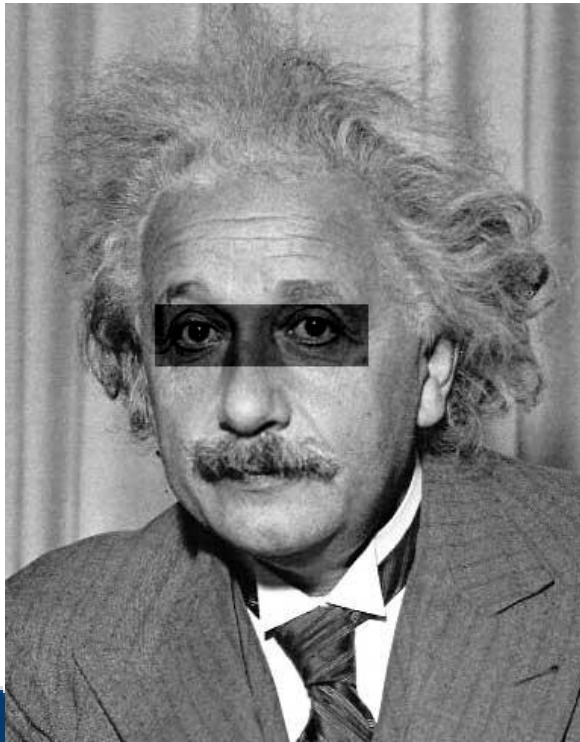


Thresholded image



Matching with filters

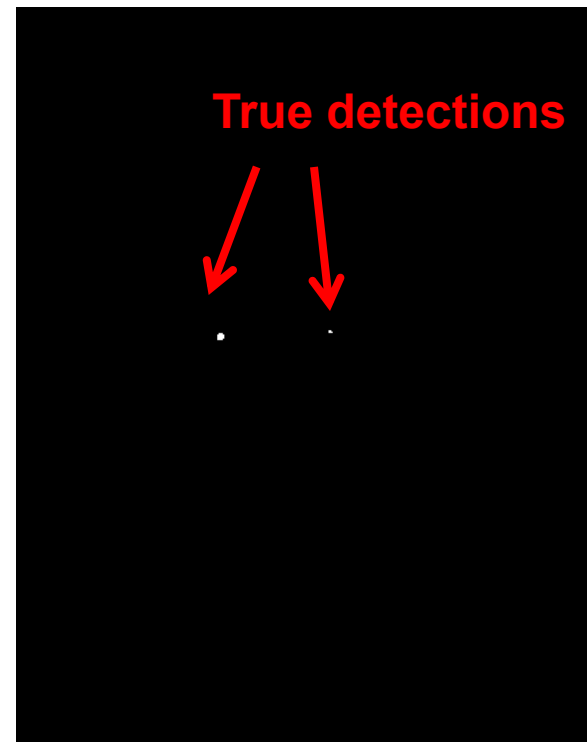
- Goal: find  in image
- Method 3: Normalized cross-correlation



Input



Normalized X-Correlation



Thresholded Image

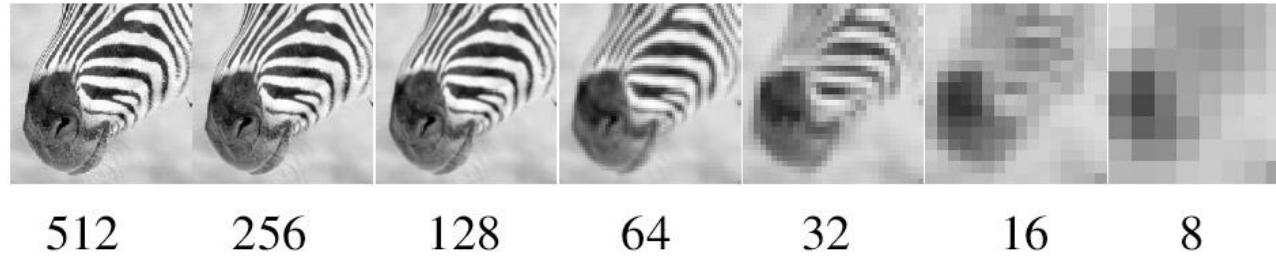


Q: What if we want to find larger or smaller eyes?

A: Image Pyramid



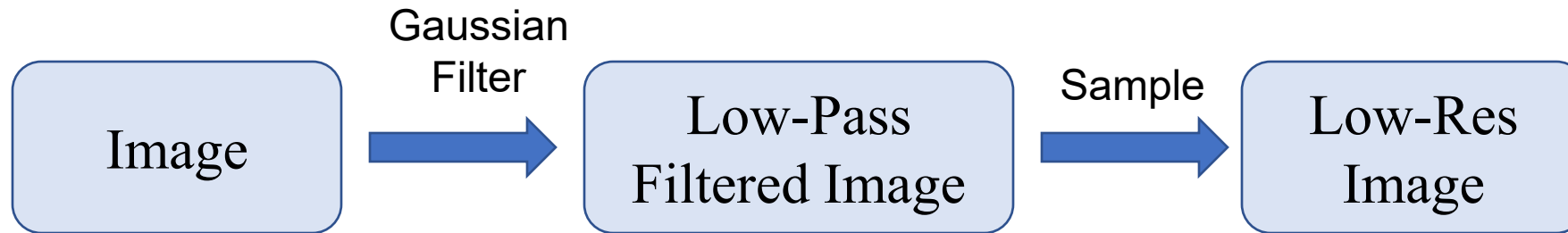
Gaussian pyramid



Source: Forsyth



Review of Sampling



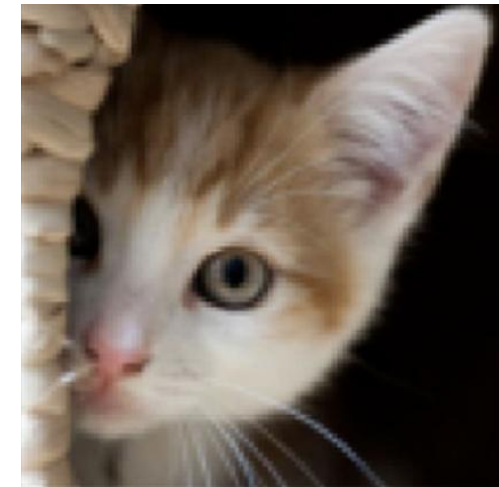
Why low-pass filter is needed?



High resolution image



Direct down-sampling



With low-pass filter



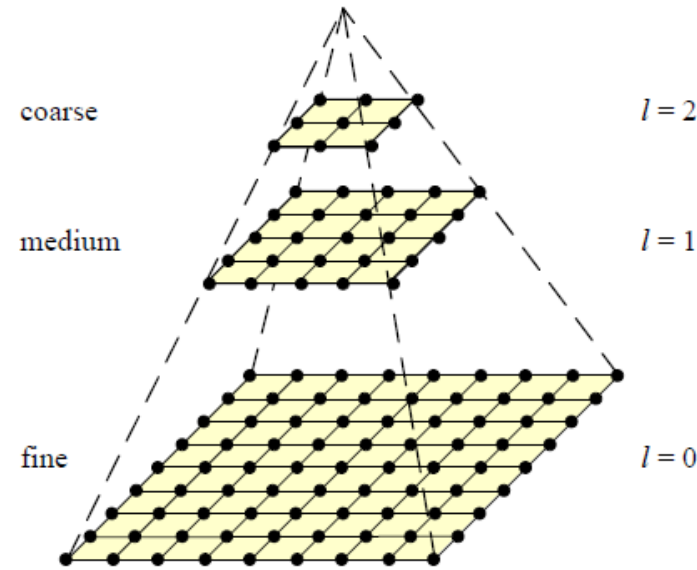
Any other benefits of Gaussian pyramid?

Speed up

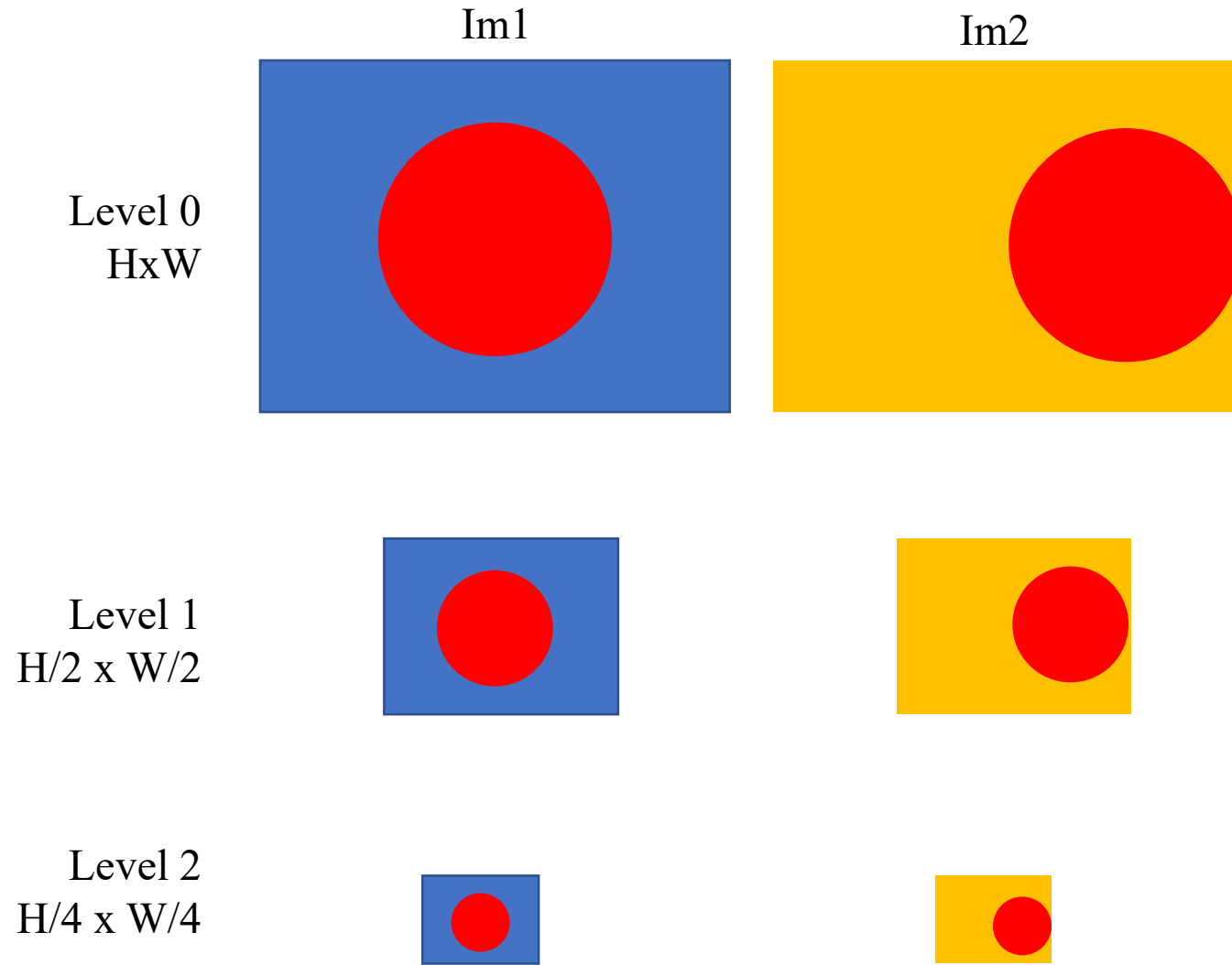


Coarse-to-fine Image Registration

1. Compute Gaussian pyramid
2. Align at coarse level
 - Find minimum SSD position
3. Successively align at finer levels
 - Search small range (e.g., 5x5) centered around position determined at coarser scale

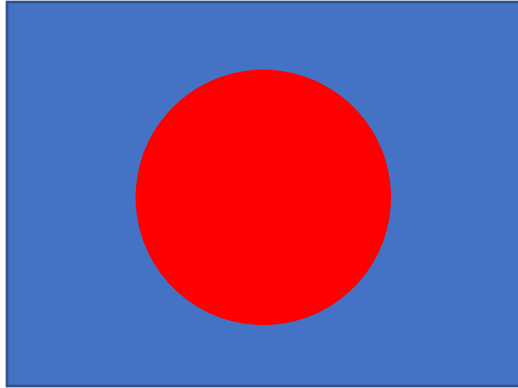


Coarse-to-fine Image Registration

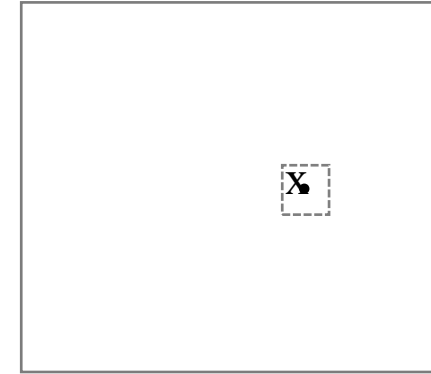
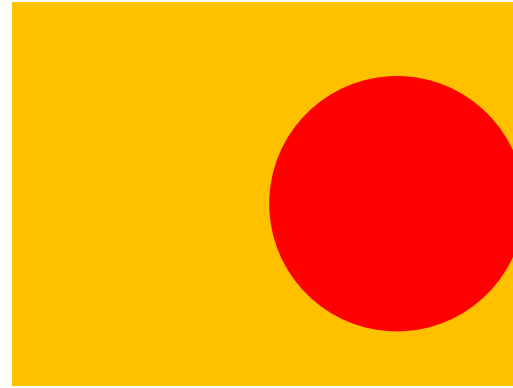


Coarse-to-fine Image Registration

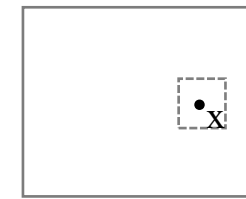
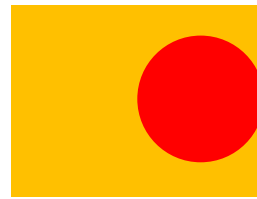
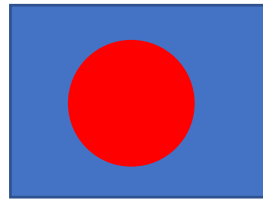
Im1



Im2



$$tx_0, ty_0 = \underset{tx \in 2 \cdot tx_1 + \{-s..s\}, ty \in 2 \cdot ty_1 + \{-s..s\}}{\operatorname{argmin}} \operatorname{SSD}(im1_0, \operatorname{translate}(im2_0, tx, ty))$$



$$tx_1, ty_1 = \underset{tx \in 2 \cdot tx_2 + \{-s..s\}, ty \in 2 \cdot ty_2 + \{-s..s\}}{\operatorname{argmin}} \operatorname{SSD}(im1_1, \operatorname{translate}(im2_1, tx, ty))$$

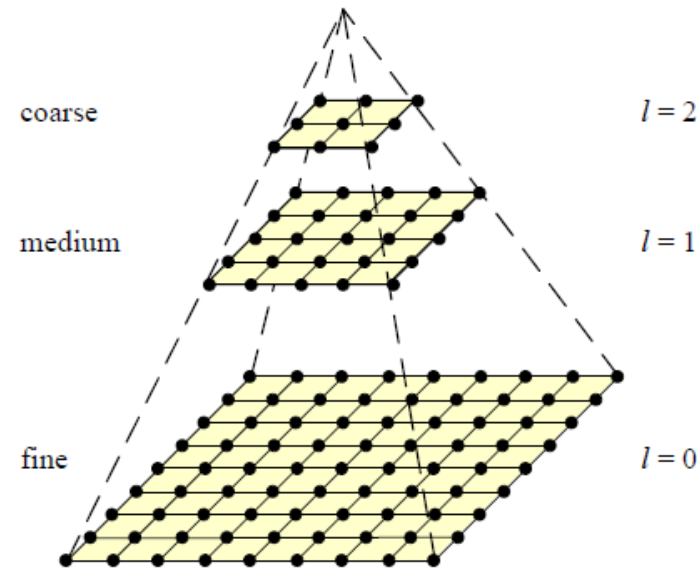


$$tx_2, ty_2 = \underset{tx \in \left\{-\frac{W}{8}.. \frac{W}{8}\right\}, ty \in \left\{-\frac{H}{8}.. \frac{H}{8}\right\}}{\operatorname{argmin}} \operatorname{SSD}(im1_2, \operatorname{translate}(im2_2, tx, ty))$$



Coarse-to-fine Image Registration

1. Compute Gaussian pyramid
2. Align at coarse level
 - Find minimum SSD position
3. Successively align at finer levels
 - Search small range (e.g., 5x5) centered around position determined at coarser scale



Why is this faster?

Are we guaranteed to get the same result?

Not all the time...

It may fail when your template has many important high-frequency information.

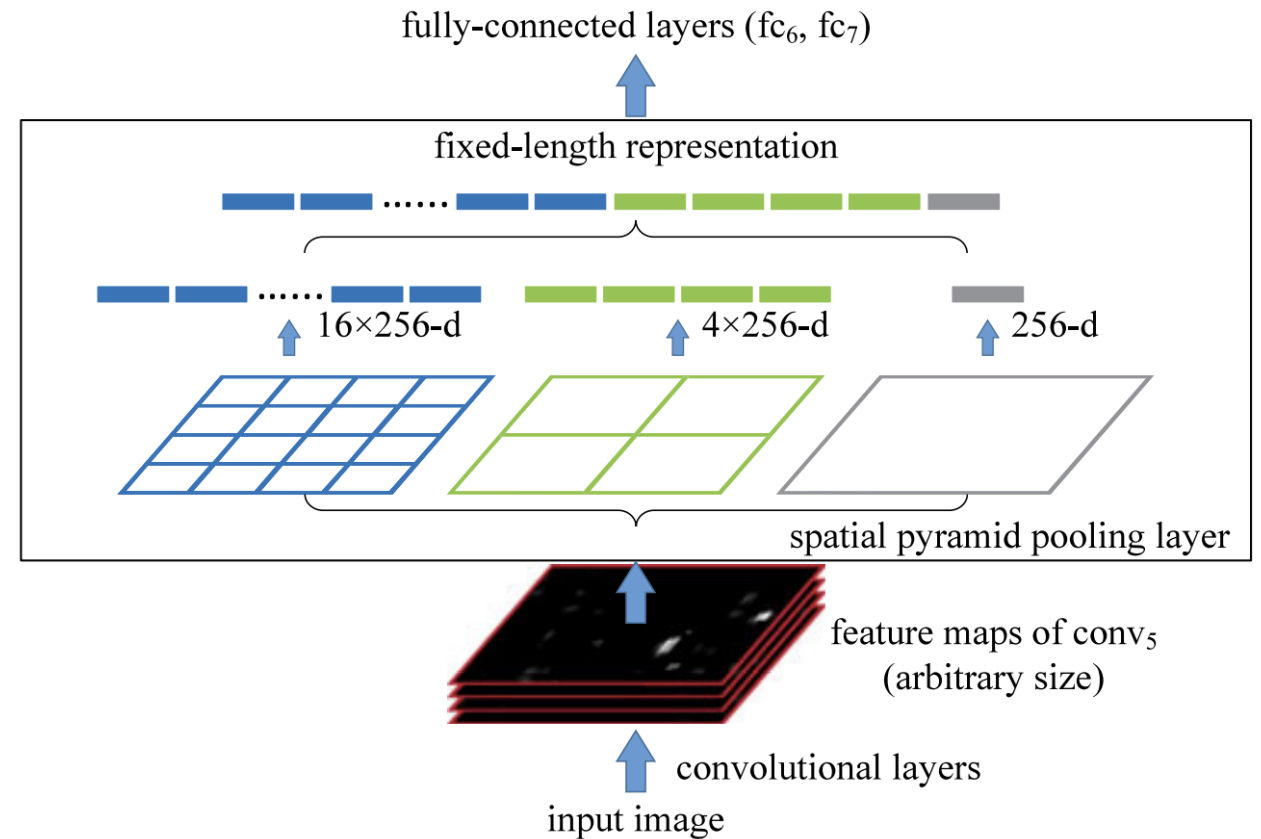
It may perform better when your image has noise.

...



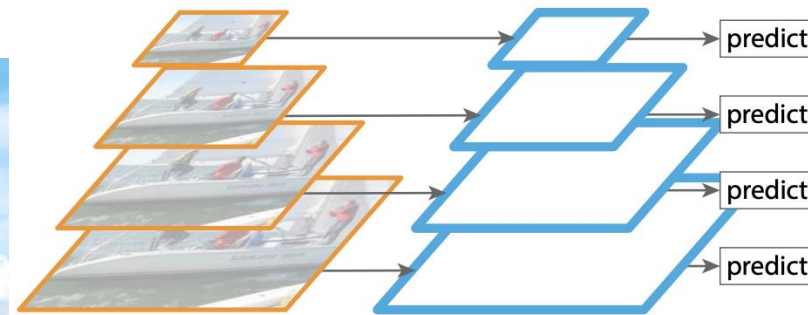
More about image pyramid

Object detection and spatial pyramid pooling

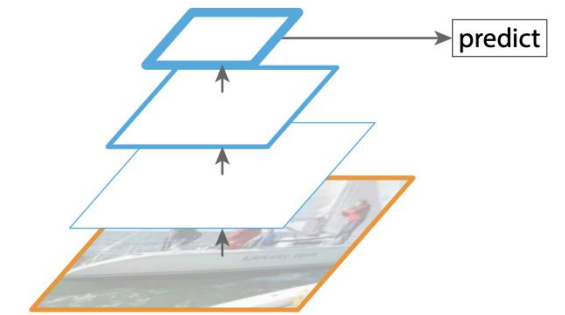


More about image pyramid

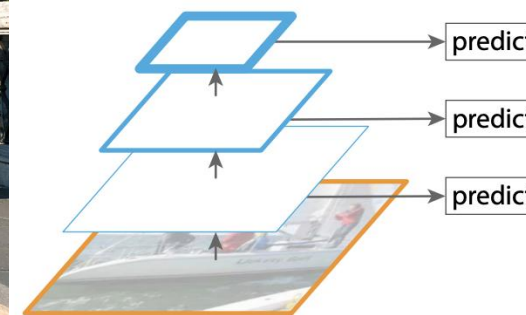
Object detection and feature pyramid networks



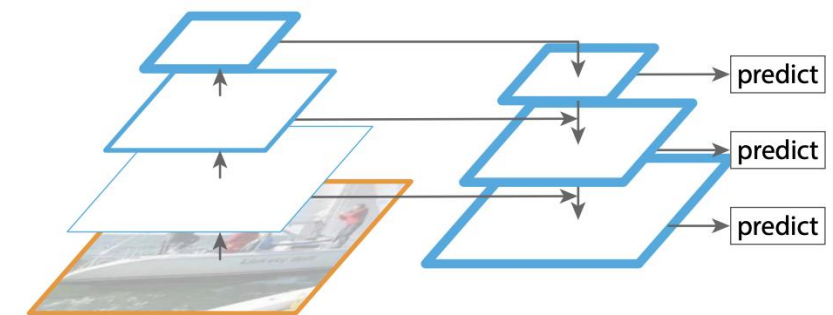
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy

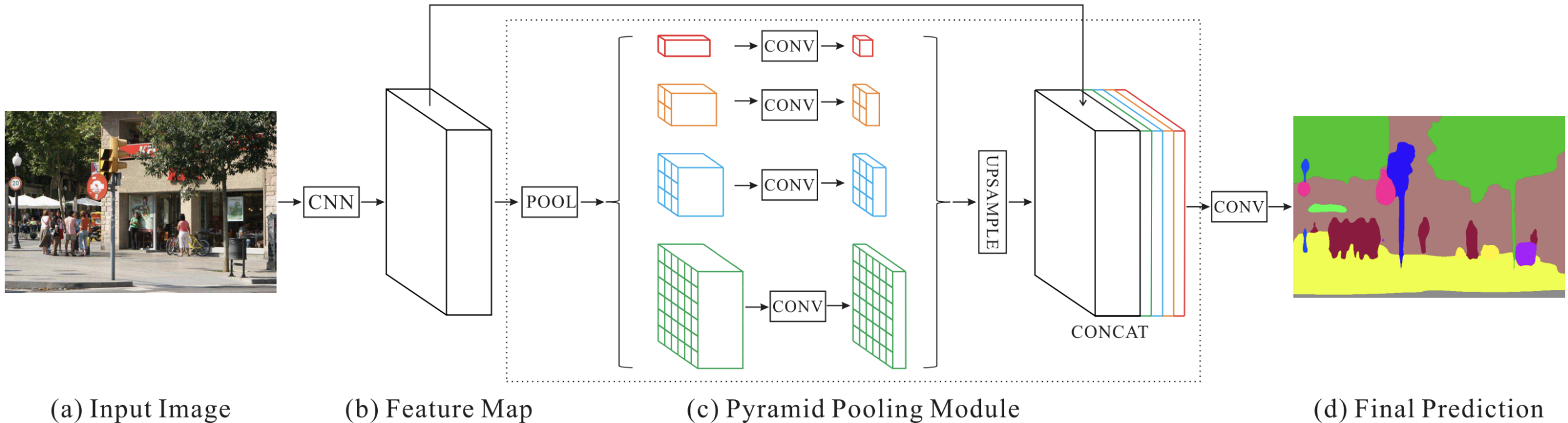


(d) Feature Pyramid Network

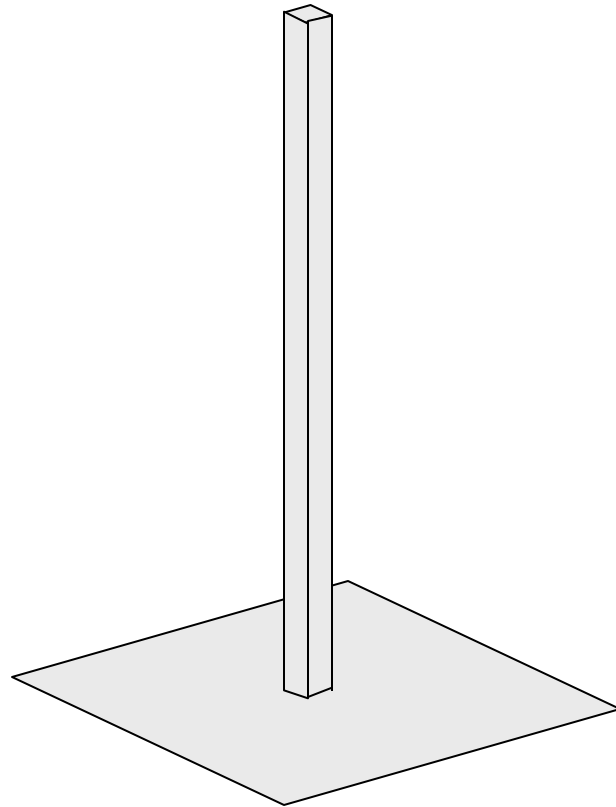


More about image pyramid

Image segmentation and image pyramid



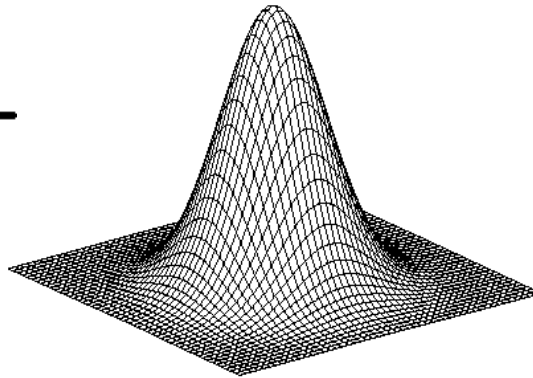
Laplacian filter



unit impulse

Smoothed areas

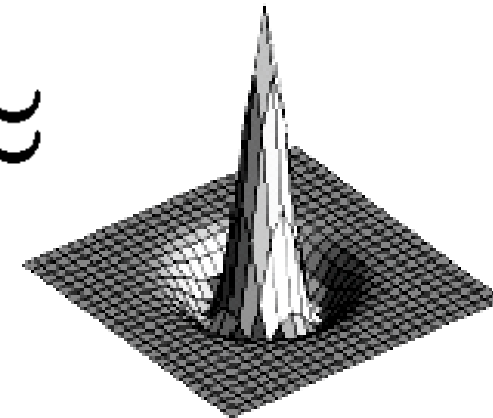
—



Gaussian

Sharped areas, such as edges

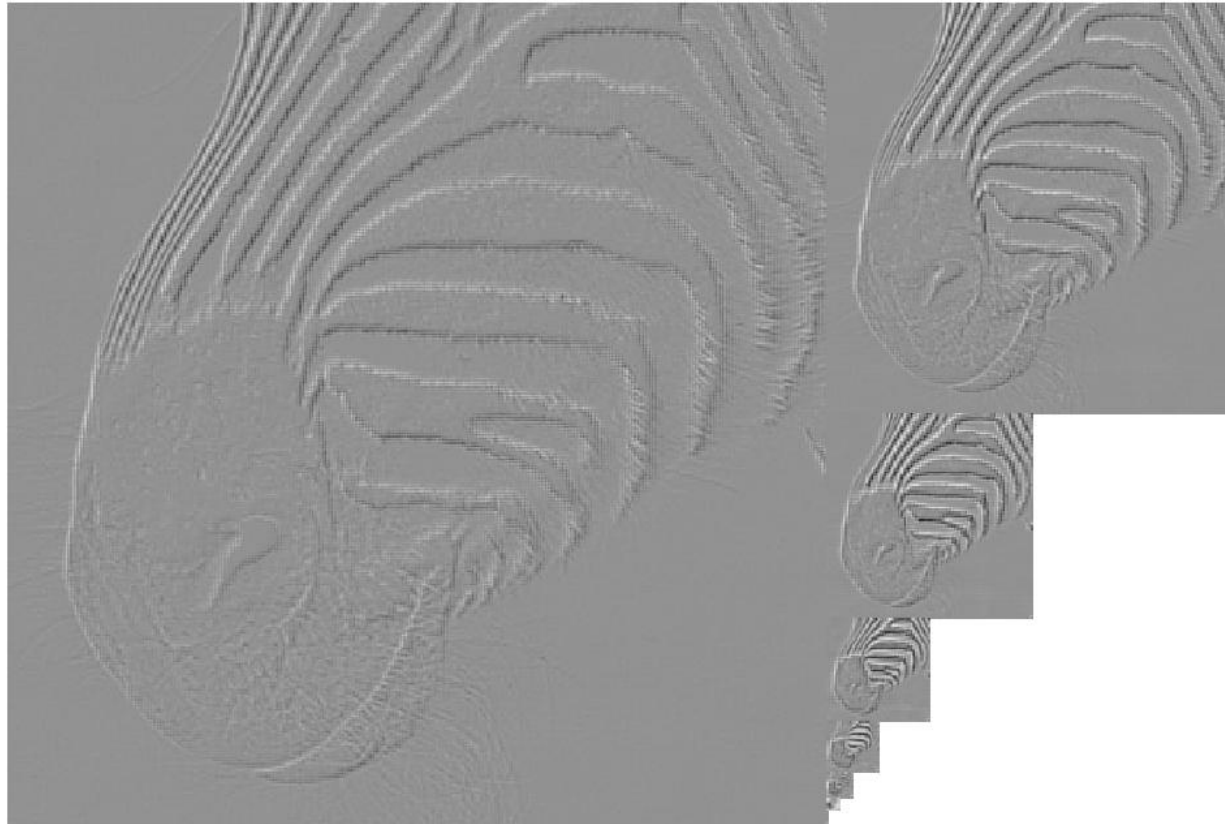
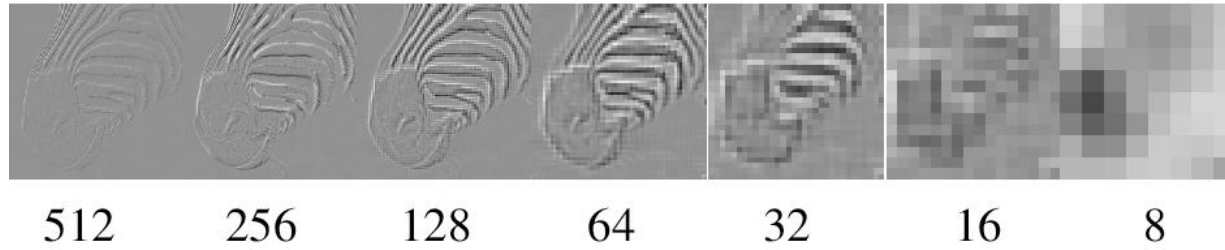
$\approx$



Laplacian of Gaussian



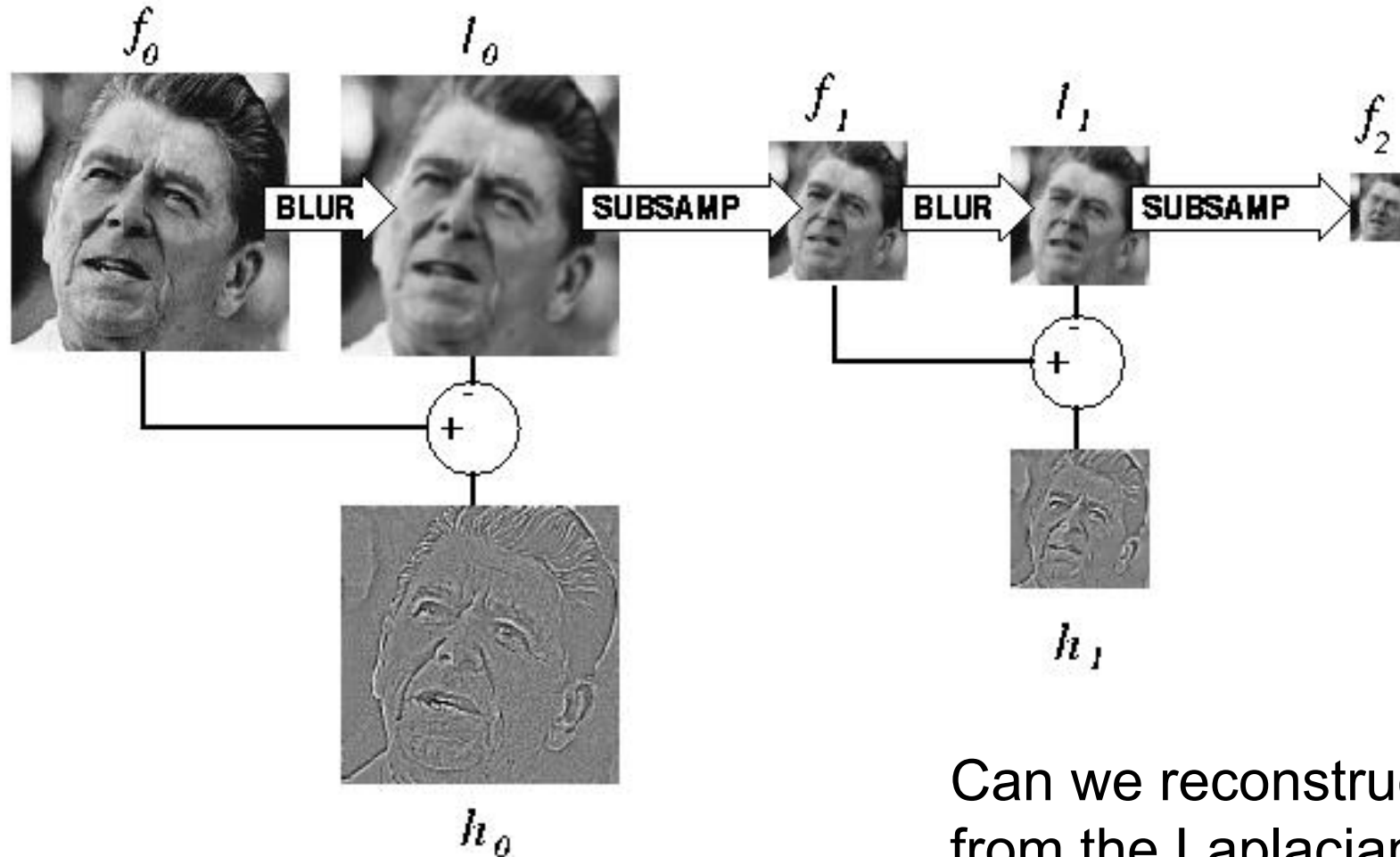
Laplacian pyramid



Source: Forsyth

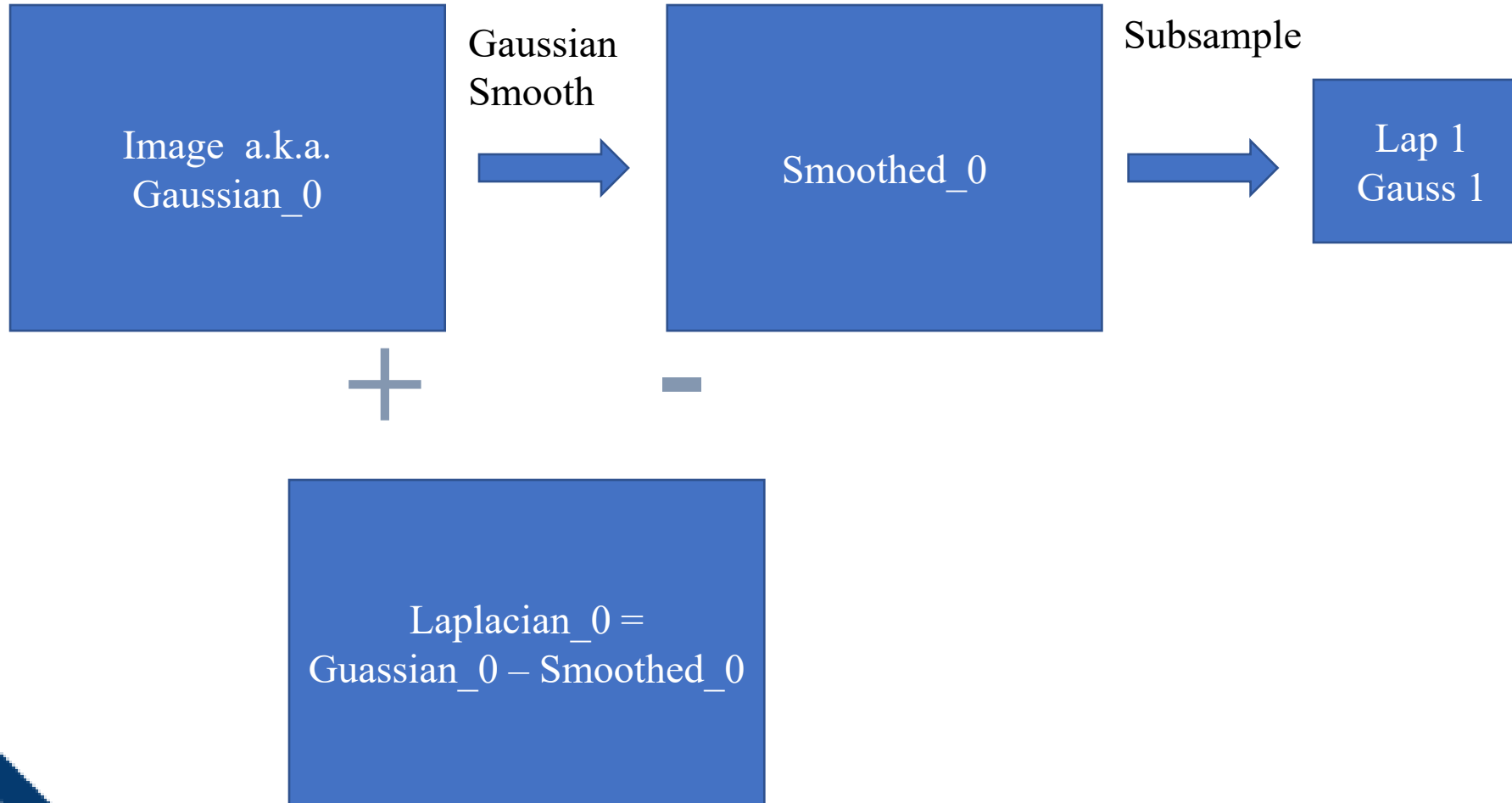


Computing Gaussian/Laplacian Pyramid

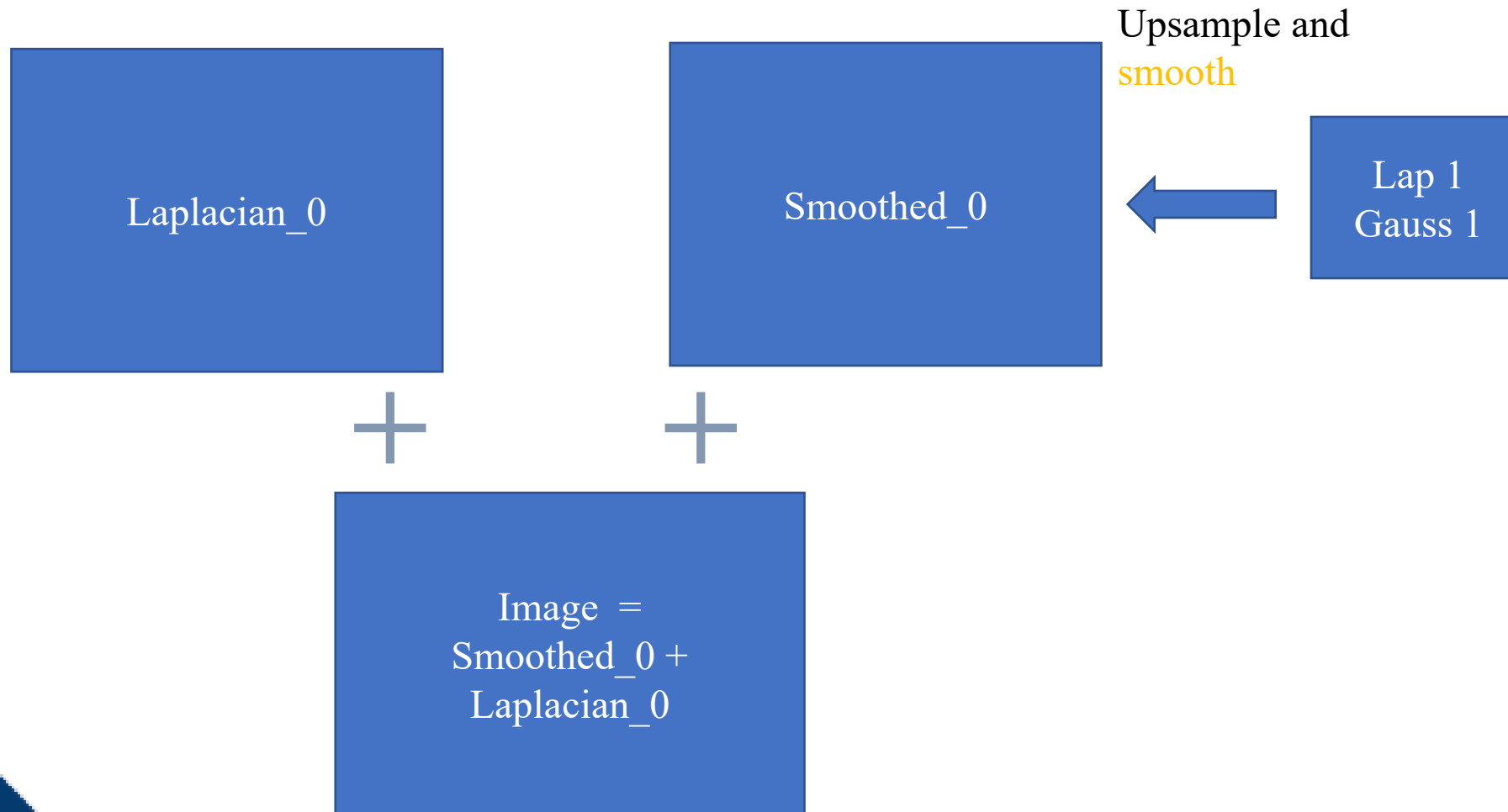


Can we reconstruct the original from the Laplacian pyramid?

Creating a 2-level Laplacian pyramid

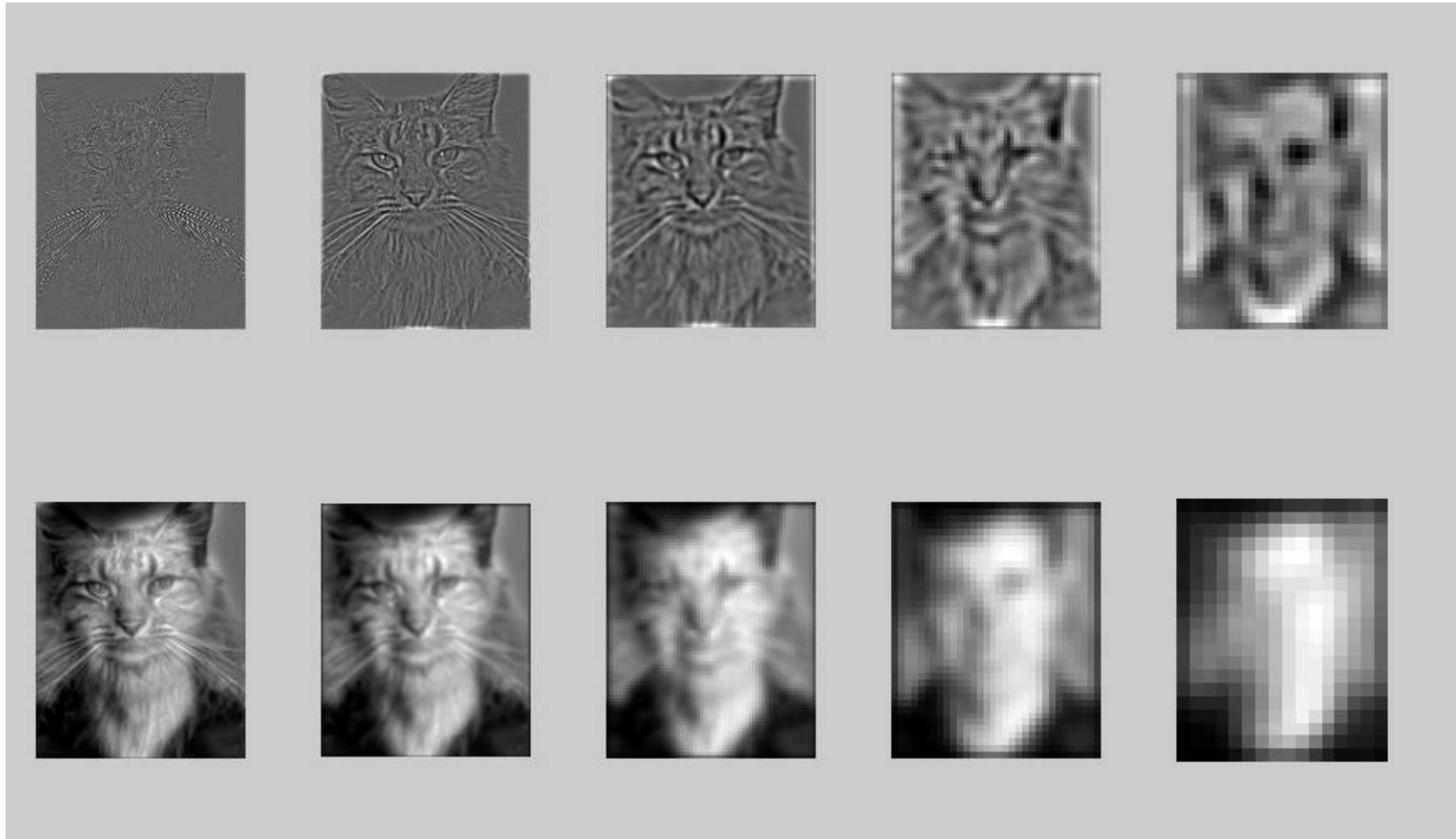


Reconstructing the image from Laplacian pyramid

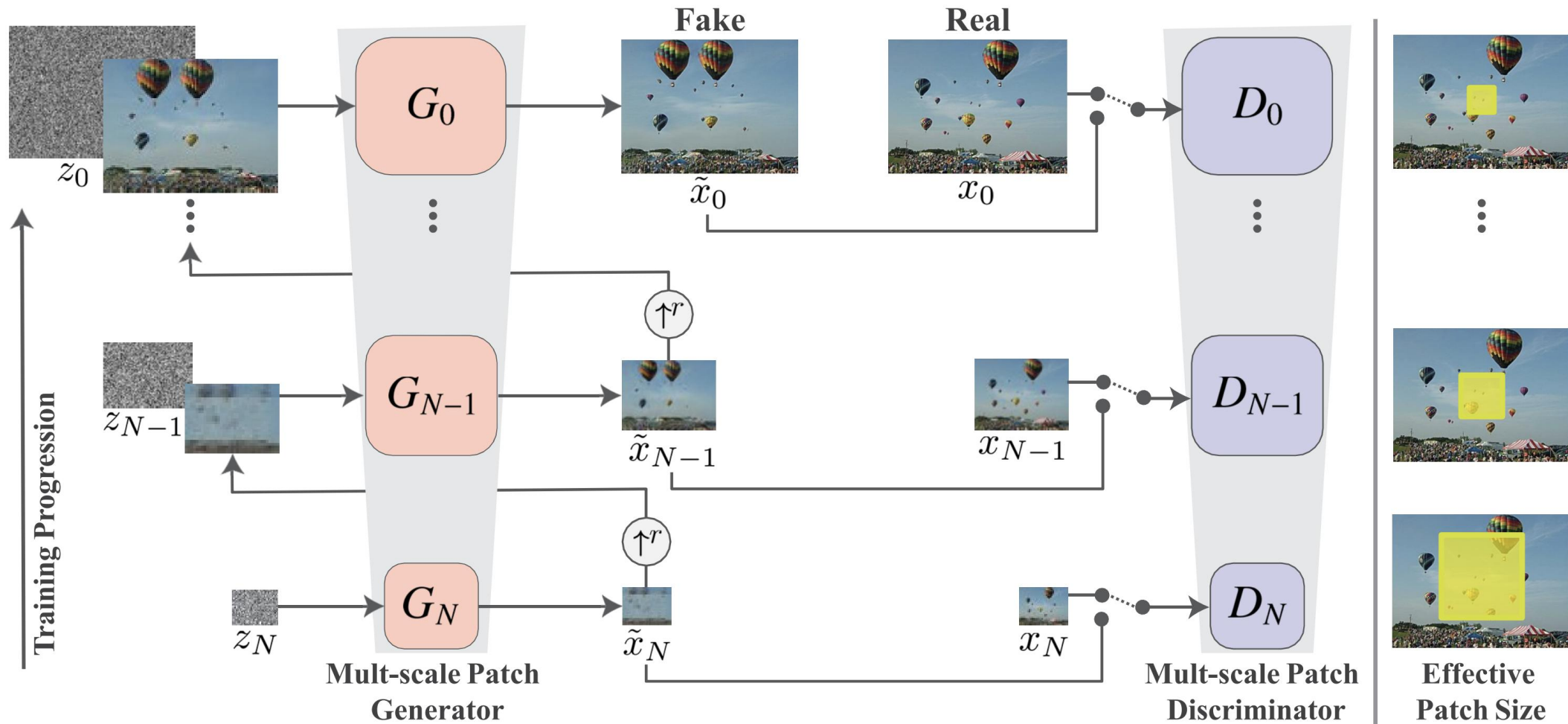


Hybrid Image in Laplacian Pyramid

High frequency $\rightarrow$ Low frequency



Similar idea about laplacian pyramid



<https://youtu.be/mdAcPe74tZI?t=3191>



Take-home messages

- Image filters can be used for
 - Denoising
 - Image enhancement
 - Template matching
 - ...
- Gaussian pyramid enables matching across different resolutions and improves the speed
- By making small modification, Gaussian pyramid becomes Laplacian pyramid, which is quite useful image synthesis applications.



**Anonymous
Class Feedback**

